

Sebastian Vintoñuke – 106063

Cindy Teresa Hsieh – 108051

Maria Delfina Cano Ros Langrehr – 109338

Lautaro Martin Sotelo – 107472

Toda afirmación que se realice debe estar apoyada en **evidencia**. Si se encuentra alguna **afirmación** que no está correctamente **fundamentada** con **evidencia**, se disminuirá la nota. Esto incluye las observaciones que realicen sobre el código y su funcionamiento en general. Que no existan métricas para un determinado QA no implica que no pueda evidenciarse el efecto de una táctica.

TODO: correr un corrector ortografico

1. SISTEMA RECIBIDO

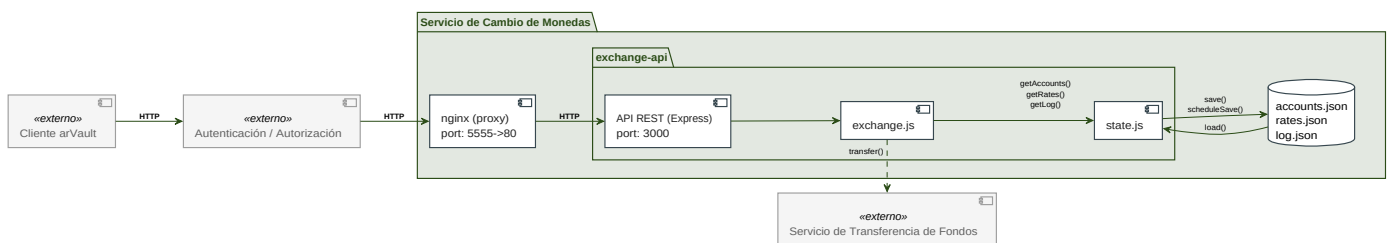


Figura 1. Diagrama Components & Connectors del sistema base.

1.1. Arquitectura

El despliegue recibido sigue una arquitectura cliente-servidor. Nginx actúa como reverse proxy y deriva las solicitudes hacia una única instancia de la API de intercambio implementada con Node.js y Express. El despliegue también incluye Graphite/StatsD, Grafana y cAdvisor como componentes de observabilidad, aunque estos no forman parte de la lógica de negocio.

Dentro de la API existe una separación lógica en tres módulos: `app.js` expone la interfaz HTTP, `exchange.js` coordina las operaciones de cambio y `state.js` carga y persiste el estado. Esta separación es parcial: la lógica de negocio conserva referencias mutables a cuentas, tasas y logs, y modifica directamente dichas estructuras.

El archivo `docker-compose.yml` declara una sola instancia de la API y no configura réplicas, health checks ni una política de reinicio. Por lo tanto, una caída del proceso interrumpe el servicio hasta que un operador o una herramienta externa lo vuelva a iniciar. Además, la API es stateful: mantiene las cuentas, tasas y operaciones en memoria, por lo que agregar réplicas sin introducir un mecanismo de coordinación produciría estados divergentes.

La persistencia se realiza periódicamente sobre archivos JSON. Las cuentas y el log se escriben cada segundo, mientras que las tasas se escriben cada cinco segundos. En consecuencia, existe una ventana entre la modificación en memoria y su escritura en disco durante la cual una terminación abrupta puede provocar pérdida de datos. Esto no constituye un protocolo de consistencia eventual, sino una estrategia de durabilidad diferida sin garantías transaccionales.

1.2. Código

La separación de responsabilidades es incompleta. `exchange.js` calcula importes, coordina las llamadas al servicio de transferencias, valida el saldo interno, modifica las cuentas y registra el resultado. Esta concentración aumenta el acoplamiento y dificulta probar cada responsabilidad de forma aislada.

```
1 export async function exchange(exchangeRequest) {
2   ...
3   //get the exchange rate
4   const exchangeRate = rates[baseCurrency][counterCurrency];
5   //compute the requested (counter) amount
6   const counterAmount = baseAmount * exchangeRate;
7   ...
8   //all good, update balances
9   baseAccount.balance += baseAmount;
10  counterAccount.balance -= counterAmount;
11  ...
12 }
```

Código 1. app/exchange.js

`state.js` no expone directamente sus variables internas, pero sus getters retornan las mismas referencias mutables almacenadas por el módulo. Luego de la inicialización, `exchange.js` conserva esas referencias y modifica los objetos sin pasar por una operación explícita del componente de persistencia. Por ello, la abstracción es permeable: reemplazar los JSON por un repositorio o una base de datos requeriría adaptar también la forma en que la lógica de negocio lee y actualiza el estado.

```

1 let accounts;
2 let rates;
3 let log;
4
5 export async function init() {
6   await stateInit();
7
8   accounts = stateAccounts();
9   rates = stateRates();
10  log = stateLog();
11 }

```

Código 2. app/exchange.js

El endpoint de cambio también contiene una condición de carrera. La comprobación de saldo se ejecuta antes de dos operaciones asíncronas. Mientras una solicitud se encuentra suspendida en un `await`, otra puede comprobar el mismo saldo y aprobar la operación. Ambas solicitudes pueden completar los descuentos y dejar un saldo negativo. El Listado 3 muestra la sección crítica, que no está protegida por exclusión mutua, serialización ni una transacción.

```

1 // app/exchange.js (caso base)
2 if (counterAccount.balance >= counterAmount) {
3   if (await transfer(clientBaseAccountId, baseAccount.id, baseAmount)) {
4     if (
5       await transfer(counterAccount.id, clientCounterAccountId, counterAmount)
6     ) {
7       baseAccount.balance += baseAmount;
8       counterAccount.balance -= counterAmount;
9       exchangeResult.ok = true;
10      exchangeResult.counterAmount = counterAmount;
11    }
12  }
13 }

```

Código 3. Sección crítica de `app/exchange.js` en el caso base.

El registro de operaciones tampoco tiene una política de retención. Cada resultado se agrega a un arreglo en memoria y `GET /log` devuelve el arreglo completo. A medida que aumenta la cantidad de operaciones, crecen el consumo de memoria, el tamaño de la respuesta y el trabajo necesario para serializar el estado. Además, el arreglo completo se convierte a JSON y se reescribe cada segundo, por lo que el costo de cada persistencia aumenta con el historial.

```

1 // app/exchange.js (caso base)
2 log.push(exchangeResult);
3
4 // app/app.js (caso base)
5 app.get("/log", (req, res) => {
6   res.json(getLog());
7 });
8
9 // app/state.js (caso base)
10 scheduleSave(log, LOG, 1000);
11
12 function scheduleSave(data, fileName, period) {
13   setInterval(async () => {
14     await save(data, fileName);
15   }, period);
16 }

```

Código 4. Crecimiento y persistencia del log en el caso base.

La escritura utiliza directamente el archivo definitivo. No se escribe primero a un archivo temporal ni se realiza un reemplazo atómico, por lo que una interrupción durante `writeFile` puede dejar un JSON incompleto. El manejo de errores de `state.js` registra el fallo, pero no lo propaga ni permite que la API informe que el estado no pudo persistirse.

```

1 async function save(data, fileName) {
2   const filePath = path.join(__dirname, fileName);
3   try {
4     await fs.promises.writeFile(filePath, JSON.stringify(data, null, 2));
5   } catch (err) {
6     console.error(`Error writing to ${filePath}:`, err);

```

```

7   }
8 }

```

Código 5. app/state.js

Los importes monetarios se representan mediante `Number`, cuyo formato de punto flotante binario no representa exactamente todos los valores decimales. Sin una regla explícita de redondeo o una representación en unidades mínimas, el error puede acumularse en los saldos y en los importes entregados al cliente.

```

1 const exchangeRate = rates[baseCurrency][counterCurrency];
2 //compute the requested (counter) amount
3 const counterAmount = baseAmount * exchangeRate;

```

Código 6. Cálculo monetario con `Number` en app/exchange.js.

Finalmente, las validaciones HTTP comprueban únicamente la presencia mediante condiciones de truthiness. No se validan tipos, rangos, monedas soportadas ni coherencia entre moneda y cuenta. Por ejemplo, un importe negativo supera la validación inicial y puede alterar los saldos en el sentido contrario al esperado. Las monedas inexistentes también pueden producir excepciones al acceder a una tasa o cuenta no encontrada. A su vez, los rechazos de negocio se informan como HTTP 500, aunque no necesariamente representan una falla interna del servidor.

```

1 // app/app.js (caso base)
2 if (
3   !baseCurrency ||
4   !counterCurrency ||
5   !baseAccountId ||
6   !counterAccountId ||
7   !baseAmount
8 ) {
9   return res.status(400).json({ error: "Malformed request" });
10 }

```

Código 7. Validación por presencia en app/app.js.**Cuadro 1.** Hallazgos del análisis estático del caso base.

Hallazgo	Evidencia	QA afectados	Consecuencia posible	Validación propuesta
Carrera concurrente	La verificación de saldo precede a dos operaciones asíncronas y el descuento ocurre después (Listado 3).	Reliability, Security	Dos solicitudes pueden aprobarse con el mismo saldo y producir un balance negativo.	Ejecutar dos cambios concurrentes de 80 unidades contra un saldo interno de 100 y observar respuestas y saldo final.
Estado mutable expuesto	Los getters de <code>state.js</code> retornan objetos que <code>exchange.js</code> conserva y modifica.	Modificabilidad, Testability, Reliability	Las actualizaciones eluden una interfaz de repositorio y dificultan sustituir o simular la persistencia.	Probar la lógica con un repositorio simulado y registrar qué dependencias deben reemplazarse.
Log sin límite	Cada operación ejecuta <code>log.push</code> ; GET /log retorna el arreglo completo (Listado 4).	Performance, Scalability, Availability	Aumentan memoria, tamaño de respuesta y tiempo de serialización con cada operación.	Ejecutar lotes crecientes y medir memoria, bytes y latencia de GET /log.
Reescritura completa	<code>scheduleSave</code> serializa y escribe el log completo cada segundo.	Performance, Efficiency, Reliability	El costo de I/O crece con el historial y una interrupción puede dejar el archivo incompleto.	Medir tamaño, tiempo de escritura y actividad de I/O para distintos volúmenes de log.
Durabilidad diferida	Los cambios residen en memoria hasta el siguiente intervalo y la escritura no es atómica.	Reliability, Availability	Una terminación abrupta puede perder operaciones confirmadas o impedir cargar el JSON.	Terminar la API dentro de la ventana de persistencia, reiniciarla y comparar estado y log.
Dinero con <code>Number</code>	El importe se obtiene mediante <code>baseAmount * exchangeRate</code> sin redondeo explícito (Listado 6).	Reliability, Security	Pueden aparecer residuos decimales y acumularse diferencias en los saldos.	Utilizar valores como 0.1 y 0.2 y comparar contra un cálculo decimal exacto.
Validación incompleta	La API solo verifica truthiness y acepta valores negativos o combinaciones incoherentes (Listado 7).	Security, Reliability, Testability	Entradas inválidas pueden alterar saldos o provocar excepciones no controladas.	Crear casos con importes negativos, tipos incorrectos, monedas inexistentes y cuentas incompatibles.

1.3. Infraestructura

La infraestructura se define mediante Docker Compose. Nginx publica la API en el puerto 5555; Grafana, Graphite/StatsD y cAdvisor conforman el subsistema de observabilidad. La API no publica directamente su puerto al host, por lo que las pruebas externas deben atravesar Nginx, tal como requiere el enunciado.

El estado de la API se copia dentro de la imagen y no está asociado a un volumen. Por lo tanto, eliminar o recrear el contenedor descarta las modificaciones de sus archivos JSON. Grafana sí utiliza un volumen, mientras que Graphite no declara uno para conservar sus series. Estas decisiones limitan la durabilidad y la reproducibilidad de las mediciones entre recreaciones del entorno.

TODO: Agregar evidencia de ejecución sobre recuperación, persistencia y funcionamiento de cAdvisor en el equipo de pruebas.

2. ATRIBUTOS DE CALIDAD

2.1. Identificación de los QA claves

Se identifican los siguientes atributos de calidad como claves en el contexto del negocio. Adicionalmente, se citan especificaciones y requerimientos que motivaron estas afirmaciones.

2.1.1. Availability (disponibilidad)

Habilidad del sistema para reparar o enmascarar fallas de manera tal que el período fuera de servicio no exceda un determinado valor durante un lapso de tiempo especificado.

La startup arVault es una fintech que opera una billetera digital

Al ser una billetera digital, el servicio permite manipular el dinero de los usuarios. Un servicio con baja o nula disponibilidad implicaría que los usuarios no pueden acceder a su propio dinero. Adicionalmente, el modelo de negocio se basa en que los usuarios operen, si el servicio no es disponible para permitir estas operaciones, se están perdiendo potenciales ganancias.

2.1.2. Performance

Habilidad del sistema para reaccionar ante ciertos eventos en un determinado tiempo.

Si bien el lanzamiento fue exitoso, comenzaron a aparecer reclamos de los usuarios sobre problemas en el uso de la función de cambio de monedas.

Uno de los principales candidatos a explicar los reclamos es el endpoint /exchange. Una operación exitosa realiza dos transferencias secuenciales de entre 200 y 400 milisegundos cada una, por lo que su tiempo de respuesta esperado es mayor que el de los endpoints que solo consultan memoria. Esta hipótesis debe contrastarse con las pruebas de carga.

las tasas de cambio no tienen gap entre el valor de compra y de venta.

La ausencia de gap significa que la compra y la venta utilizan la misma cotización; no implica por sí misma que la tasa se actualice en tiempo real. Sin embargo, una latencia elevada aumenta el tiempo entre la cotización y la finalización del cambio, y por ello incrementa el riesgo de operar con un valor que ya no representa las condiciones vigentes.

2.1.3. Scalability (escalabilidad)

Habilidad del sistema para incorporar recursos adicionales de manera efectiva. Efectividad = mejora mensurable de algún QA + no requerir un esfuerzo inapropiado + no interrumpir el funcionamiento.

Para ganar usuarios, las tasas de cambio...

Se especifica que el objetivo a corto plazo es incrementar el número de usuarios. El sistema debe poder acompañar ese crecimiento incorporando recursos sin interrumpir el servicio ni exigir un esfuerzo inapropiado.

2.1.4. Reliability (confiabilidad)

Grado en que la arquitectura es susceptible a fallas a nivel de sistema cuando se producen fallas parciales en componentes, conectores o datos.

La confiabilidad es clave porque el servicio modifica saldos que representan dinero. Una respuesta exitosa con un saldo incorrecto constituye una falla aunque el proceso permanezca disponible. La condición de carrera y la persistencia diferida identificadas en la Sección 1.2 pueden producir pérdida de operaciones confirmadas, balances negativos o un estado distinto después de reiniciar el servicio.

2.1.5. Security (seguridad)

Capacidad del sistema de prevenir acciones maliciosas o accidentales fuera del uso esperado.

No se valida que haya saldo en la cuenta del cliente; se espera que lo haga la UI.

Confiar en validaciones realizadas exclusivamente por el cliente permite que un consumidor invoque directamente la API con valores inesperados. Aunque el enunciado supone autenticación y autorización previas, esos mecanismos no sustituyen la validación de tipos, rangos e invariantes de negocio. Una operación inválida puede provocar pérdidas monetarias y reputacionales.

2.2. Otros QA menos relevantes

Además se mencionan otros QA que, aunque no resulten claves y por ende no sean una prioridad, son deseables, especialmente conforme el proyecto crezca.

TODO: Buscar en el enunciado o en la documentación evidencia que justifique estos QA.

2.2.1. *Efficiency*

Performance por recurso consumido.

2.2.2. *Testability (testeabilidad)*

Grado en que un componente/sistema de software permite ser probado para exhibir sus fallas; usualmente relacionado al esfuerzo necesario para llevar adelante las tareas de testing.

2.2.3. *Manageability*

Define la facilidad de gestión del sistema por parte de los administradores del mismo.

2.2.4. *Simplicity*

Facilidad con la que la arquitectura de un sistema puede ser entendida, razonada y explicada.

2.3. Decisiones de diseño actuales y su impacto en los QA

2.3.1. *Impacto en la Performance*

...

2.4. Critica

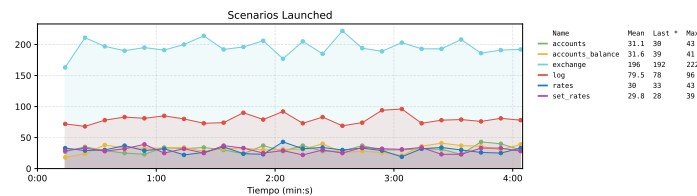
3. COMPORTAMIENTO DEL SERVICIO

3.1. Pruebas de carga

La prueba de carga se realizó usando una tasa constante de 40 usuarios durante 240s (4 minutos). Cada usuario realiza un solo request, es decir se mantiene un throughput constante de 40 requests por segundo. No se realizó warm-up ni ramp para no ensuciar las métricas de promedios. Tampoco se realizaron recargas de saldo intermedias, se utilizó una carga inicial suficiente para cubrir toda la prueba.

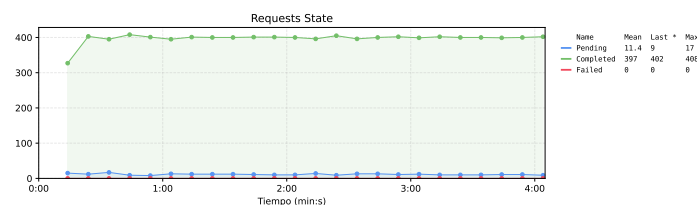
Adicionalmente, aprovechando que siendo este el sistema inicial no se plantea una comparativa, decidimos para ilustrar el formato de los gráficos y explicar los criterios compartidos entre todos los escenarios.

Se pueden encontrar los resultados completos en el [Apendice](#).



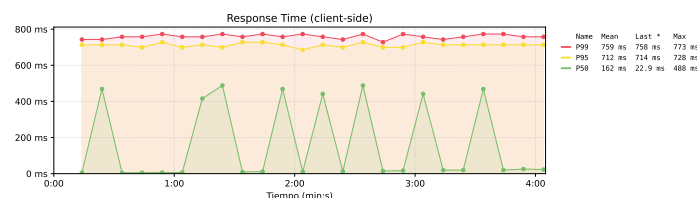
Para un primer diagnóstico del sistema se decidió utilizar todos los endpoints. Se asignaron 20 requests por segundo a exchange por ser el endpoint principal en el contexto de negocio. Se asignaron 8 requests por segundo a log por las hipótesis planteadas sobre su significativo impacto en la disponibilidad y latencia del sistema. Se asignaron 3 request por segundo al resto de endpoints. Para todos los gráficos salvo se indique lo contrario el eje x muestra el tiempo transcurrido de la prueba y cada punto agrupa una ventana de 10 segundos. Adicionalmente en las leyendas de la derecha se muestran estadísticas relevantes al escenario, como por ejemplo el promedio, el último valor y/o el máximo.

3.1.1. Disponibilidad



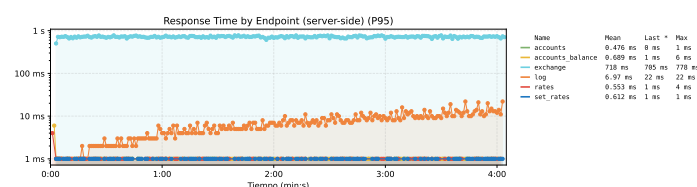
Para evaluar la disponibilidad del sistema monitoreamos el estado de las request. Completed indica cuántas requests finalizaron correctamente en la ventana de 10 segundos, mientras que Pending indica cuántas seguían en curso. En 4 minutos con una carga de 40 request por segundo, se puede observar que el sistema no presentó inconvenientes.

3.1.2. Latencia

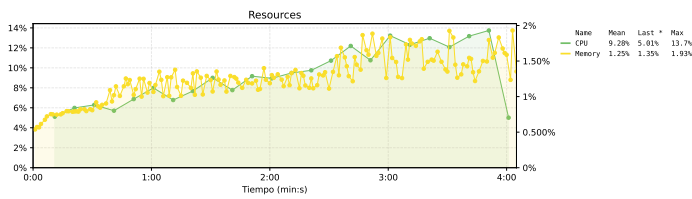


Para evaluar la latencia del sistema optamos por utilizar dos metodologías, la primera monitorear los tiempos de respuestas desde la perspectiva del cliente a través de Artillery. Optamos por realizar el monitoreo en P50, P95 Y P99, sin embargo una consideración importante para esta metodología es que no distingue entre endpoints. Una consecuencia de esto último es por ejemplo los saltos en P50 donde dependiendo si la ventana contiene mayormente requests rápidas o del endpoint exchange, la mediana se desplaza entre ambos grupos.

La siguiente metodología adoptada fue medir los tiempos de respuesta por endpoint desde la perspectiva del servidor. Se realizó agregando la recolección de métricas directamente en el servidor. Si bien en contraposición a la metodología anterior si podemos discernir entre endpoints, optamos por representarlo solo para P95 para no saturar la representación. Para el eje y optamos por una escala logarítmica por la gran discrepancia entre los tiempos. Una observación que veníamos anticipando es que los endpoints se agrupan en tres clases, exchange lento por sus dependencias externas pero constante, log que crece junto con el tamaño del propio log y los restantes los cuales son considerablemente más rápidos. En próximos escenarios nos vamos a limitar a monitorear un endpoint de cada grupo, es decir exchange, log y optamos por rates como representante de los endpoints rápidos, de esta forma simplificar la representación.

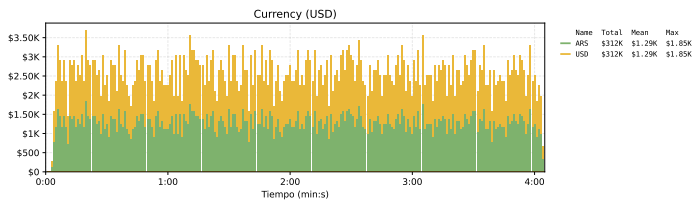


3.1.3. Uso de recursos



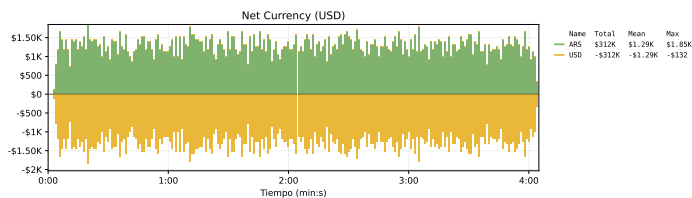
Para evaluar el uso de recursos del sistema, monitoreamos el uso del CPU y de la memoria. El uso del CPU se lee sobre el eje y al lado izquierdo mientras que el uso de la memoria al lado derecho. Nuevamente, mantiene el formato de tiempo en el eje x con ventanas de 10 segundos. Una primera observacion es que tanto el uso de CPU como el de memoria crece con el tiempo, se especula debido al crecimiento del log.

3.1.4. Volumen operado por moneda



Como la primera metrica de negocio, se evaluo el volumen operado por cada moneda. Para esto se agrego la recoleccion de metricas directamente en el servidor. Optamos por representar cada moneda con su respectivo valor en dolares, a la cotizacion vigente en cada operacion, para que los valores sean comparables. Los volúmenes de diferentes monedas operados en un mismo punto se apilan para representar el total operado en ese punto. Se mantiene el formato de tiempo en el eje x pero se redujo la ventana a 1 segundo.

3.1.5. Volumen neto operado



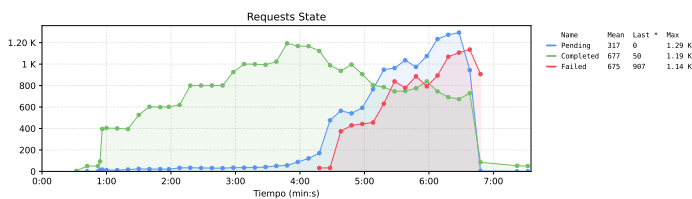
Para la segunda metrica de negocio, se evaluo el volumen neto operado por cada moneda. Nuevamente, a travez de la recoleccion de metricas directamente en el servidor. De igual modo, con cada moneda en su respectivo valor en dolares. A diferencia del volumen operado, las ventas se representan como valores negativos y las compras con valores positivos. Se mantiene el formato de tiempo en el eje x y la ventana de 1 segundo.

3.2. Pruebas de estrés

La prueba de estrés se realizo incrementando progresivamente la carga desde 40 hasta 180 en saltos de 20, cada salto con una duracion de 45s, el tiempo total de la prueba es de 450s (7,5 minutos). Se realizo un warm-up inicial de 5 usuarios durante 30s. Ademas, al final de la prueba se dejo una instancia de recovery de nuevamente 5 usuarios esta vez durante 60s. Tampoco se realizaron recargas de saldo intermedias y se mantuvo el mismo uso de endpoints para que los resultados sean comparables.

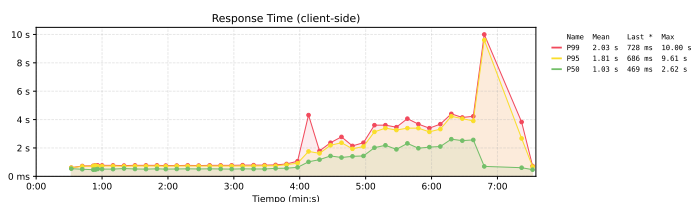
Se pueden encontrar los resultados completos en el [Apendice](#).

3.2.1. Disponibilidad

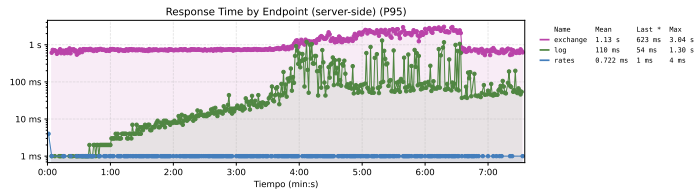


Se observa que el servidor bajo el crecimiento de la carga encuentra su punto de quiebre al rededor del minuto 4:30. A partir de ese momento, la disponibilidad del sistema se ve comprometida y se continuan completando solicitudes pero otras fallan.

3.2.2. Latencia

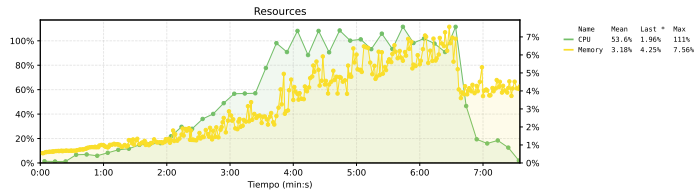


Se observa como previo a la perdida de disponibilidad antes mencionada (4:30) el servidor ya empieza a presentar una degradacion en los tiempos de respuesta desde el minuto 4.



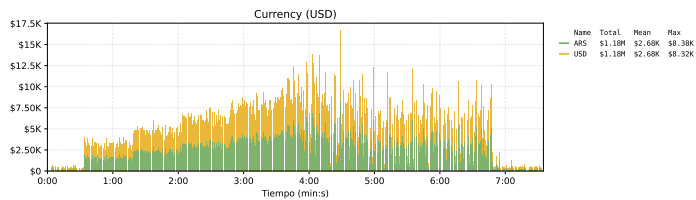
Se observa como el crecimiento en los tiempos de respuesta del log, dado por el crecimiento del mismo, impacta directamente en la latencia del endpoint de exchange.

3.2.3. Uso de recursos



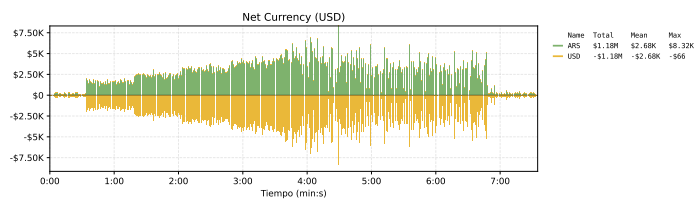
Se observa que el cuello de botella del servidor es el CPU, ya que alcanza su maxima capacidad en el minuto 4, junto con el comienzo de la degradacion del servidor. Igualmente aunque en menor medida la memoria tambien muestra signos de crecimiento. Ambos factores generados por el crecimiento del log, lo que aumenta el costo de serializacion para el CPU y el uso de memoria para almacenar los datos.

3.2.4. Volumen operado por moneda



Se observa como se degrada el volumen operado a partir de minuto 4, esto implica potenciales perdidas economicas tanto para usuarios como para la plataforma.

3.2.5. Volumen neto operado



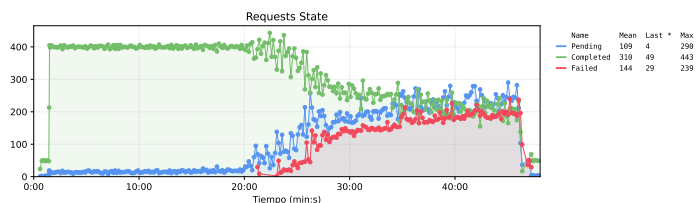
Para el volumen neto, se observan los mismo resultados expuestos para el volumen operado por moneda.

3.3. Pruebas de resistencia

La prueba de resistencia se realizo usando una tasa constante de 40 usuarios durante 2700s (45 minutos). Se realizo un warm-up inicial de 5 usuarios durante 60s. Ademas, al final de la prueba se dejo una instancia de recovery de nuevamente 5 usuarios esta vez durante 120s (2 minutos). Tampoco se realizaron recargas de saldo intermedias y se mantuvo el mismo uso de endpoints para que los resultados sean comparables.

Se pueden encontrar los resultados completos en el [Apendice](#).

3.3.1. Disponibilidad

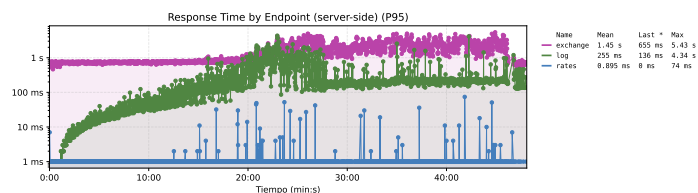


A diferencia con el escenario de estres, en este caso se observa que la perdida de disponibilidad no es producto del aumento de la carga si no que tambien se produce la misma degradacion bajo un escenario prolongado de carga continua. Las primeras request fallidas empiezan a ocurrir posterior al minuto 20. A partir de ahi se observa un incremento sostenido en la cantidad de fallos hasta la fase de recuperacion al final de la prueba.

3.3.2. Latencia

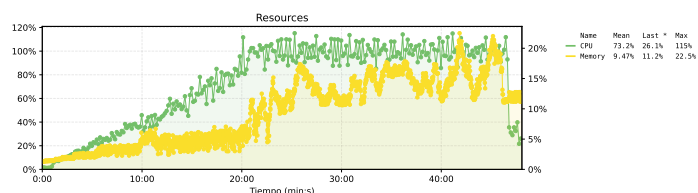


Se observa como nuevamente el aumento en la latencia desde la perspectiva del cliente es en el minuto 20, previo a la aparición de las primeras request fallidas. Coherente con una degradación progresiva y no un error abrupto.



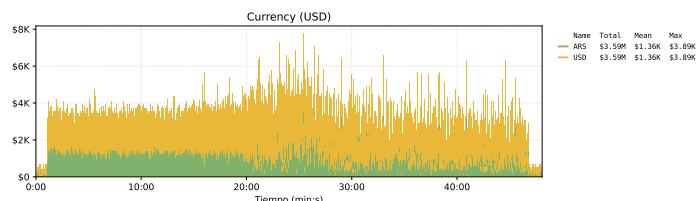
En cuanto a la latencia por endpoint se observan los mismo síntomas que los descritos en la prueba de estrés. El endpoint de log perjudica la latencia del endpoint de exchange conforme el tamaño del log aumenta y con este su tiempo de respuesta. Este comportamiento se evidencia a partir del minuto 20, consistente con las conclusiones de disponibilidad y latencia antes mencionadas.

3.3.3. Uso de recursos



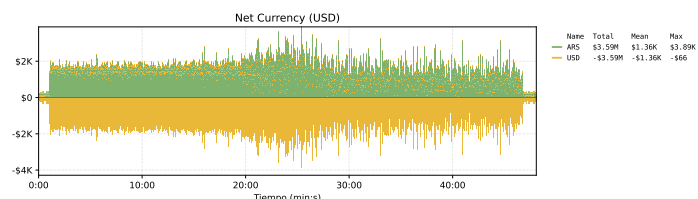
Se observa un incremento en el uso de recursos, especialmente de CPU, alcanzando este el máximo disponible a los 20 minutos. El uso de memoria también siguió incrementando con respecto a las observaciones en las pruebas de carga, llegando hasta un máximo de 22.5 % de la memoria disponible.

3.3.4. Volumen operado por moneda



Se observa una degradación en el volumen operado a partir de los 20 minutos con respecto a los minutos previos.

3.3.5. Volumen neto operado



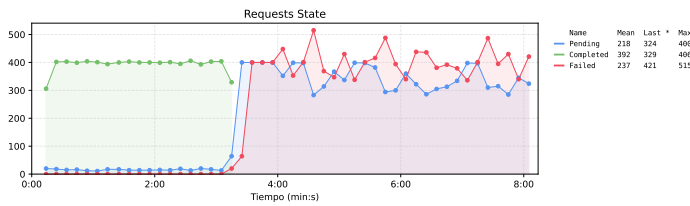
Se observa una tendencia similar en el volumen neto operado, con una disminución notable a partir de los 20 minutos.

3.4. Pruebas de caos

La prueba de carga se realizó usando una tasa constante de 40 usuarios durante 480s (8 minutos). El reparto de request se mantiene igual que en la prueba de resistencia. Adicionalmente se agrega un request malintencionado sabiendo que no hay un buen manejo de errores. Este tiene una probabilidad suficiente para aparecer entre 2 o 3 veces a lo largo de la prueba. Cada usuario realiza un solo request, es decir se mantiene un throughput constante de 40 requests por segundo. No se realizó warm-up ni ramp porque no aportaban información relevante para esta prueba. Tampoco se realizaron recargas de saldo intermedias, se utilizó una carga inicial suficiente para cubrir toda la prueba.

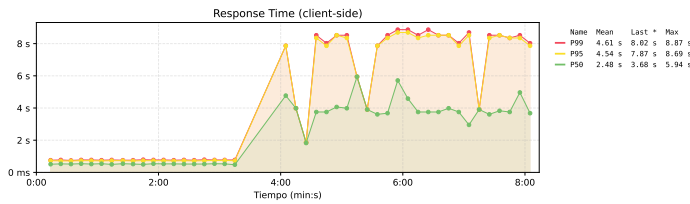
Se pueden encontrar los resultados completos en el [Apendice](#).

3.4.1. Disponibilidad

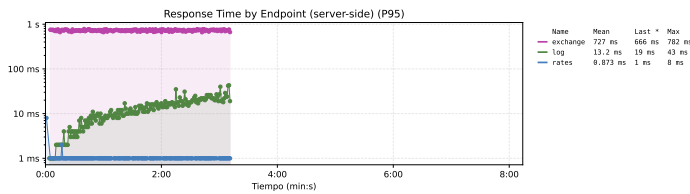


Se observa como el sistema cae cerca del minuto 3:15 el la primera request invalida. A partir de ese momento las request no se completan y esperan un timeout completo de 10s. Se verifica que el sistema no se recupera como se menciona en la Sección 1.1.

3.4.2. Latencia

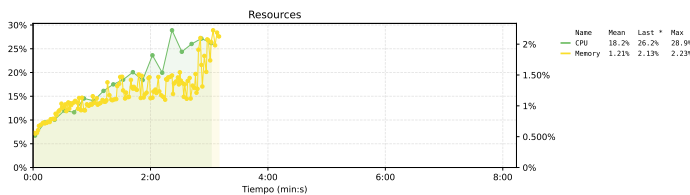


Posterior a la caída se observa el aumento en los tiempos de respuesta siendo estas devueltas por nginx como errores 502 en lugar de por el propio servidor.



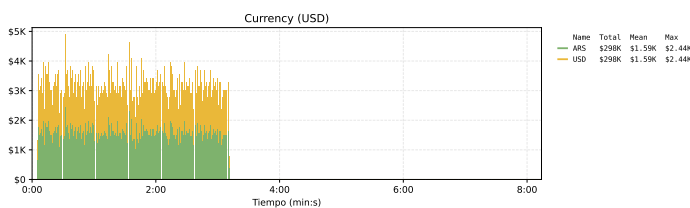
No se observan metricas posteriores al momento de la caída. Esto se debe a que el mecanismo para recolectar las metricas es parte del propio servidor y deja de funcionar cuando el servidor cae.

3.4.3. Uso de recursos



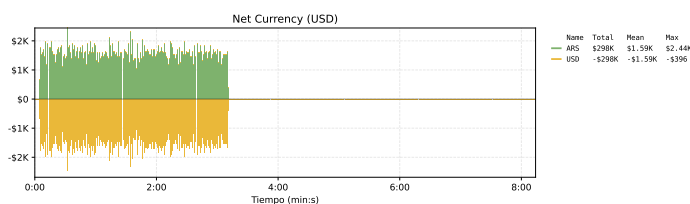
De igual manera, se dejan de reportar las métricas de uso de recursos posterior a la caída del servidor. Esto se debe a la caída del propio proceso.

3.4.4. Volumen operado por moneda



Posterior a la caída y como consecuencia de esta se deja de operar, el total se estanca en \$298K por moneda en 8 minutos.

3.4.5. Volumen neto operado



Analogamente, posterior a la caída y como consecuencia de esta se deja de operar y el volumen neto se estanca.

3.5. Resultados y conclusiones

Los resultados obtenidos del escenario de carga nos indican que podemos clasificar los endpoints en tres categorias lo que nos permitira simplificar nuestro analisis. Adicionalmente, los resultados del escenario de estres y resistencia nos indican que el sistema presenta una

degradación significativa por la mala gestión del log. Esta observación motivará la propuesta 1. Finalmente, los resultados del escenario de caos nos indican que el sistema base no provee mecanismos de recuperación. Esta observación motivará la propuesta 2.

4. PROPUESTA 1

El sistema presenta una degradación principalmente por el manejo indevido del log. Destacamos que la posibilidad de permitir consultar el log de manera completa es la principal causa de degradación. Este escala con el número de operación, lo que provoca un aumento en el uso de recursos para mantenerlo, y un aumento en los tiempos de respuesta para consultarlo. Esto arrastra la degradación a los otros endpoints, entre ellos exchange, el principalmente mencionado por los usuarios.

Destacamos que un endpoint que permite consultarlo todo por definición no es escalable, recomendamos la implementación de paginación. Sin embargo, asumimos que esta es un requerimiento de negocio y que las mejoras propuestas no deben alterar la firma del sistema para con los usuarios. Se propone desacoplar el log para que la degradación de este no afecte a los endpoints de negocio del sistema. Proponemos agregar un nodo dedicado, análogo a exchange-api para el manejo del log. Algunas optimizaciones de la implementación que consideramos que vale la pena mencionar son:

- Reemplazamos el uso del formato .json para el almacenamiento a .ndjson, de esta manera podemos agregar los registros al final sin reescribir todo el archivo.
- El archivo ya no se carga en memoria, se streamea el cliente directamente desde disco en batches. Destacamos que agregamos los caracteres [al inicio y] al final del stream para mantener la firma original con respecto al formato .json.
- Solo enviamos hasta el último byte válido disponible al momento de recibida la consulta y verificamos que la última entrada sea válida (evita race condition con la escritura).
- Exchange api loguea directamente al nuevo nodo dedicado mediante una conexión TCP dedicada.

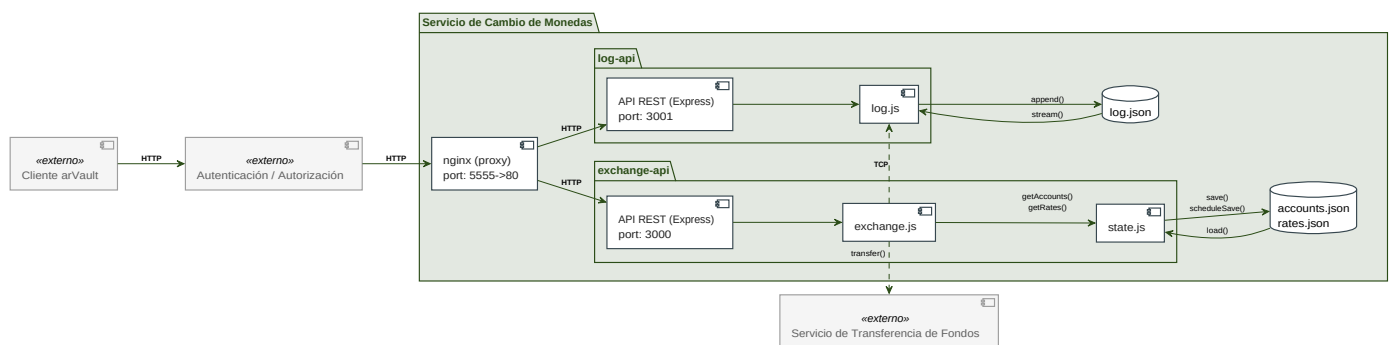


Figura 2. Diagrama Components & Connectors de la propuesta 1.

4.1. Impacto en los QA

4.1.1. Performance

TODO: evaluar el impacto en cada QA

4.2. Pruebas de carga

Las pruebas de carga comparten la misma configuración con el sistema base de tal forma que los resultados sean comparables, ver Pruebas de carga.

Se pueden encontrar los resultados completos en el [Apendice](#).

4.3. Pruebas de estrés

Las pruebas de estrés comparten la misma configuración con el sistema base de tal forma que los resultados sean comparables, ver Pruebas de estrés.

Se pueden encontrar los resultados completos en el [Apendice](#).

4.3.1. Disponibilidad

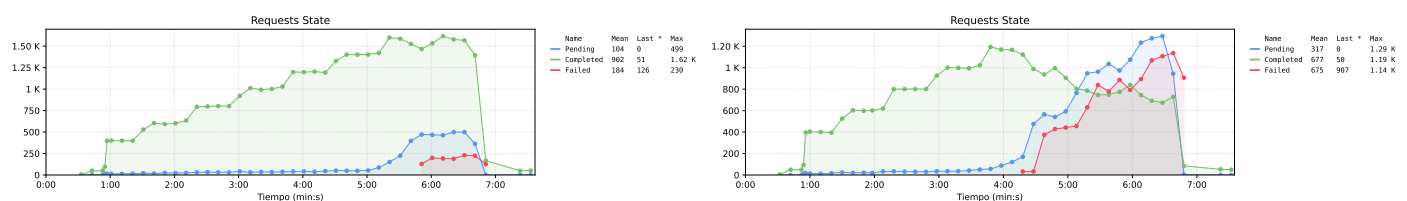


Figura 3. Comparativa de la disponibilidad ofrecida por la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. La propuesta 1 obtuvo un incremento en el promedio de respuestas satisfactorias de 225 respuestas adicionales, lo que representa un aumento del 33, 23 % frente al caso base.

4.3.2. Latencia

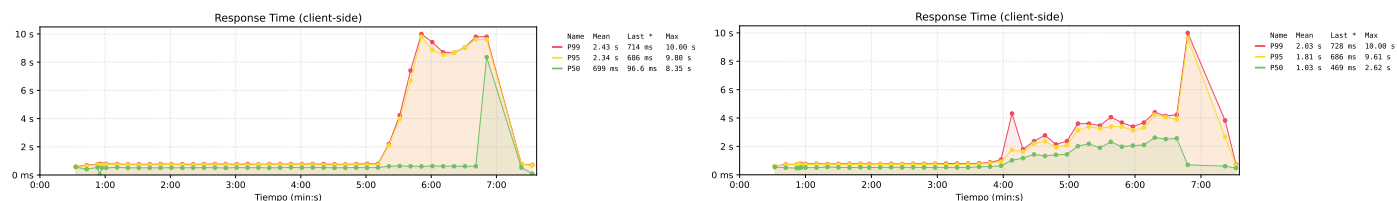


Figura 4. Comparativa de los resultados de latencia entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Se observa que la propuesta 1 mejoro el promedio de latencia para P50. Tambien se observa que empeoro en P95 y P99.

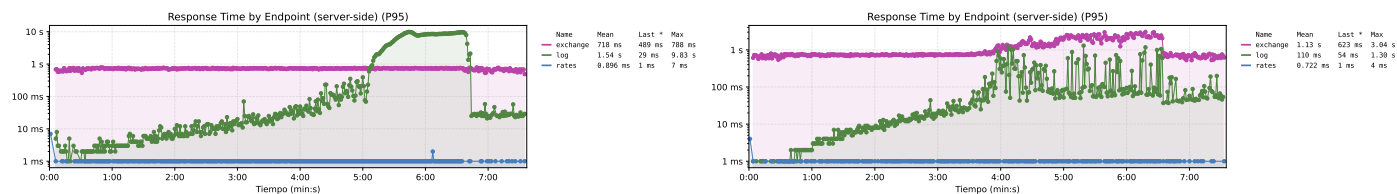


Figura 5. Comparativa de los resultados de latencia por endpoint entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Con diferencia el analisis mas interesante, la propuesta 1 permitio como se esperaba mejorar la latencia en el endpoint de exchange, se puede notar que este no se ve afectado por la degradacion del endpoint de logs. Como si ocurría en el caso base, donde al rededor del minuto 4, los tiempos de respuesta de exchange incrementaban, en este caso, los tiempos de respuesta se mantienen estables en 1 segundo, acotado por la dependencia del servicio externo. Por otro lado, los tiempos de respuesta de /log empeoraron, sinembargo es esperable, la mayor disponibilidad antes mencionada implica que se registraron mas transacciones, por lo que el tamaño del log necesariamente aumento. Esto explica el aumento de los tiempos de respuesta visto desde el lado del cliente antes mencionados, sinembargo, las metricas se ven ensuciadas al mezclar todos los endpoints.

4.3.3. Uso de recursos

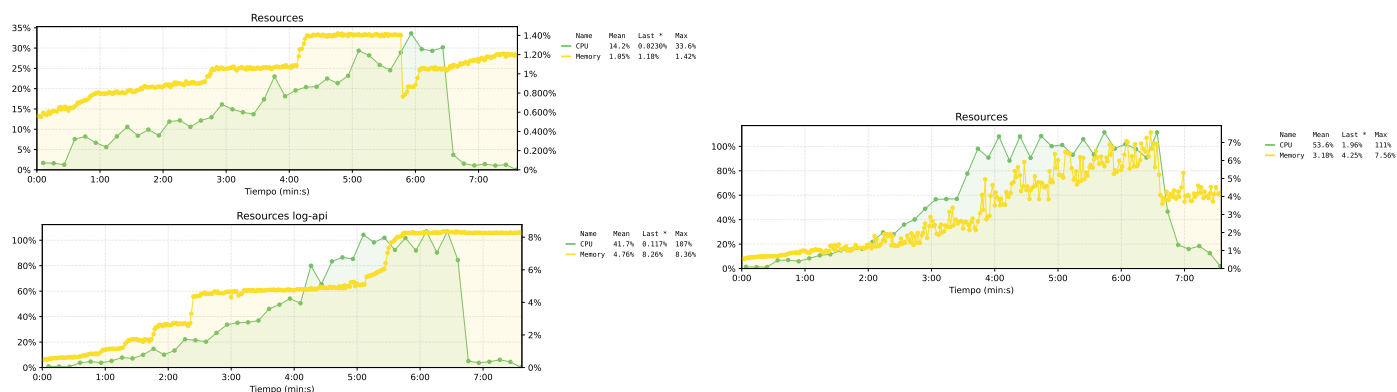


Figura 6. Comparativa de los resultados del uso de recursos entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Destacar que consideramos la comparacion con respecto a ambos nodos, el nodo de exchange y el de logs. Se puede observar que el nodo de logs mantiene la carga con respecto a la del sistema base, incluso la aumenta. Esto es esperable con el aumento de la disponibilidad, mas request completadas implican mas entradas en el log. Sinembargo, el nodo de exchange apenas llevo a un 14.2 % de uso de CPU y 1.42 % de uso de memoria.

4.3.4. Volumen operado por moneda

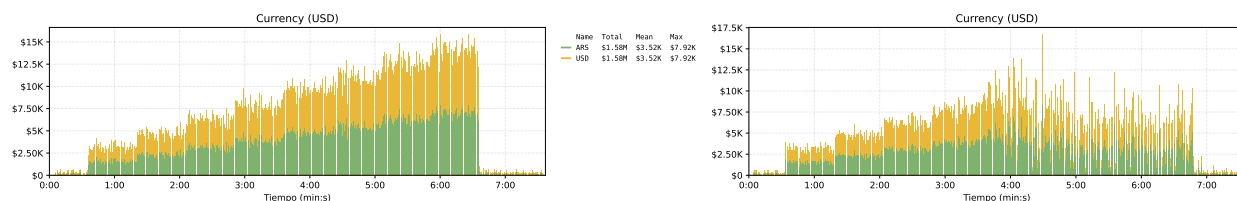


Figura 7. Comparativa de los resultados de volumen operado en cada moneda entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Se observa que la propuesta 1 obtuvo un mayor volumen total siendo este 1.58M frente al 1.18M del sistema base, es decir una mejora del 33.90 %.

4.3.5. Volumen neto operado

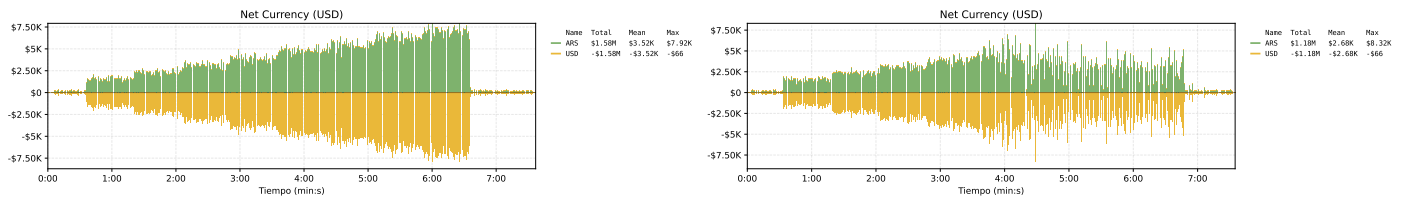


Figura 8. Comparativa de los resultados de volumen neto operado en cada moneda entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de estrés. Analogamente al volumen operado, se observa que el volumen neto también es mayor en la propuesta 1 frente al sistema base.

4.4. Pruebas de resistencia

Se pueden encontrar los resultados completos en el [Apendice](#).

4.4.1. Disponibilidad

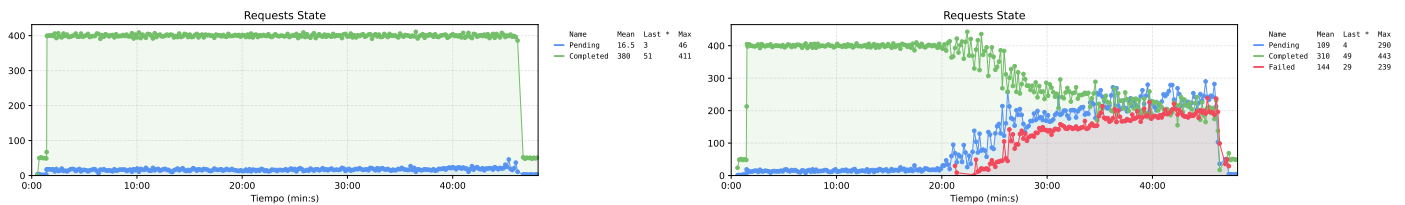


Figura 9. Comparativa de la disponibilidad ofrecida por la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia. El sistema base mostraba una degradación crítica a los 20 minutos, mientras que la propuesta 1 logra completar satisfactoriamente todas las request durante los 45 minutos.

4.4.2. Latencia

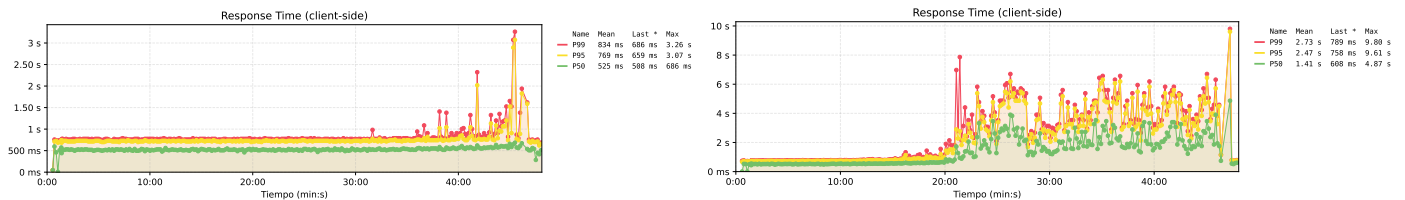


Figura 10. Comparativa de los resultados de latencia entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia. Consistente con los resultados de disponibilidad se muestra una gran mejora especialmente posterior al minuto 20. Sin embargo, la propuesta 1 sigue mostrando una principio de degradación a los 40 minutos, esperable dado el crecimiento del log.

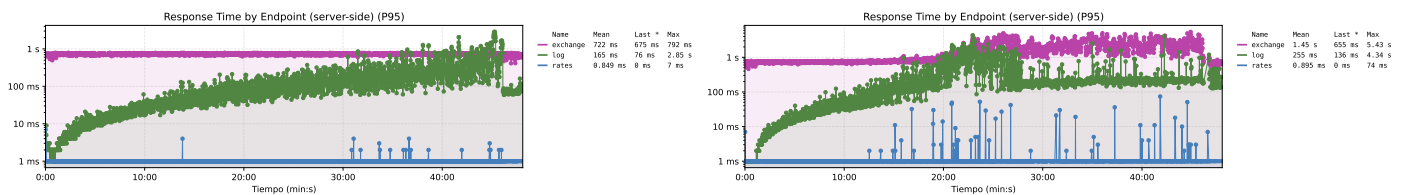


Figura 11. Comparativa de los resultados de latencia por endpoint entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia. Para el tiempo de respuesta desde la perspectiva del servidor, se muestra como en la propuesta 1 los tiempos del endpoint de logs ya no “empujan” los tiempos del endpoint exchange, sin embargo como ya se mencionó, el endpoint de logs continúa degradando. Los tiempos de respuesta de log entre ambos casos lamentablemente no son comparables puesto que menor disponibilidad implica un menor crecimiento del propio log.

4.4.3. Uso de recursos

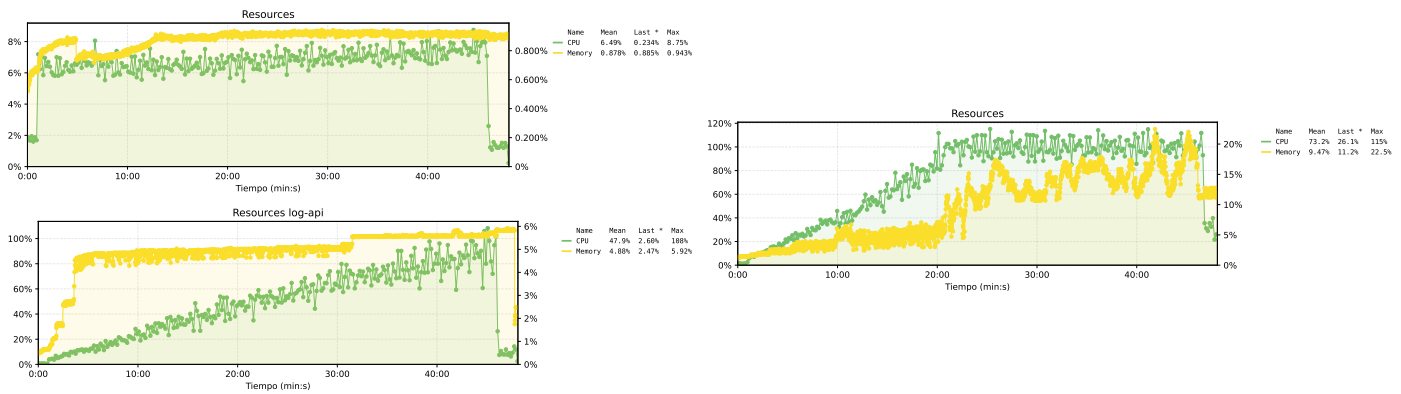


Figura 12. Comparativa de los resultados del uso de recursos entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia. Destacar que ambos casos no son directamente comparables, la propuesta 1 dispone de dos contenedores/nodos independientes cada uno con los mismo recursos que el nodo monolítico del escenario base. Se observa que para el nodo de exchange-api los recursos son prácticamente ociosos con apenas un uso del 6.4 % de CPU y 0.8 % en memoria. También se observa en los recursos del nodo de logs, un uso más constante de la memoria, esperable al streamear los datos desde disco y no conservar todos los logs en memoria.

4.4.4. Volumen operado por moneda

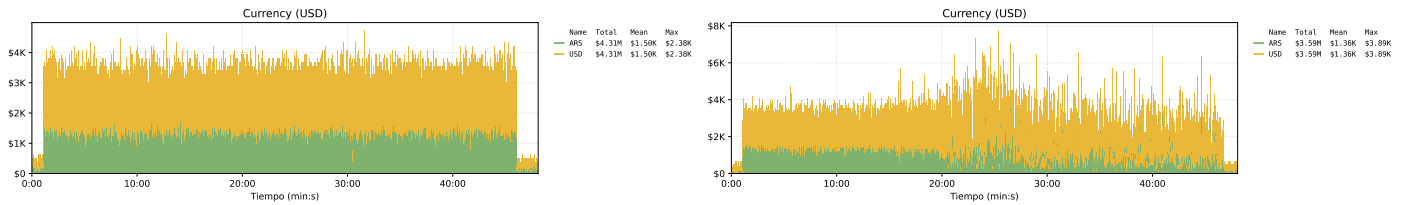


Figura 13. Comparativa de los resultados de volumen operado en cada moneda entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia. Se observa que la propuesta 1 obtuvo un mayor volumen total siendo este 4.31M frente al 3.59M del sistema base, es decir una mejora del 20.06 %.

4.4.5. Volumen neto operado

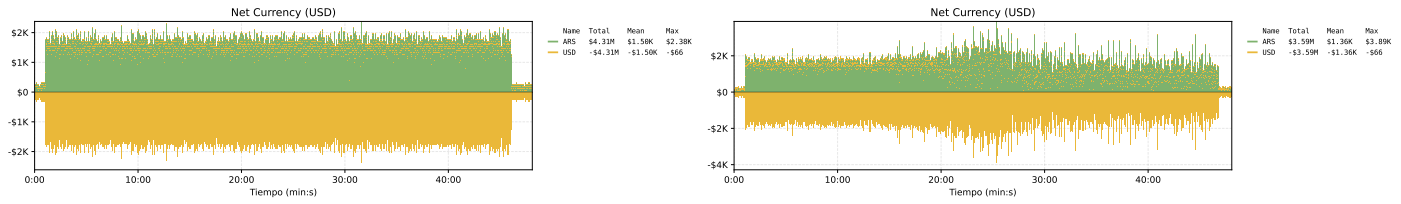


Figura 14. Comparativa de los resultados de volumen neto operado en cada moneda entre la propuesta 1 (izquierda) y el sistema base (derecha) frente a un escenario de resistencia. Análogamente al volumen operado, se observa que el volumen neto también es mayor en la propuesta 1 frente al sistema base en la misma proporción.

4.5. Resultados y conclusiones

La propuesta 1 demostro ser capaz de aislar la degradacion producida por la consulta de logs de las opeeraciones de negocio del sistema, permitiendo ofrecer una mejor disponibilidad a los usuarios comunes. Por otro lado la consulta de logs sigue degradandose con el tiempo. Sinembargo, este desacoplamiento propuesto permitiria replicar el nodo de consulta para dar soporte de lectura de los a mas usuarios, es decir, permitiendo escalabilidad horizontal.

5. PROPUESTA 2

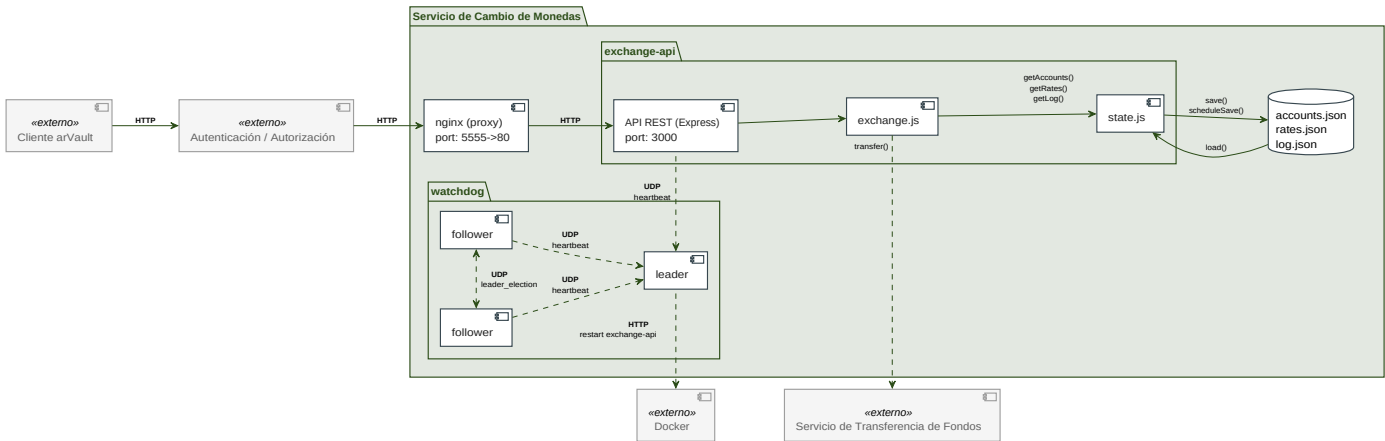


Figura 15. Diagrama Components & Connectors de la propuesta 2.

5.1. Impacto en los QA

5.1.1. Performance

5.2. Pruebas de carga

Se pueden encontrar los resultados completos en el Apéndice.

5.3. Pruebas de estrés

Se pueden encontrar los resultados completos en el Apéndice.

5.3.1. Disponibilidad

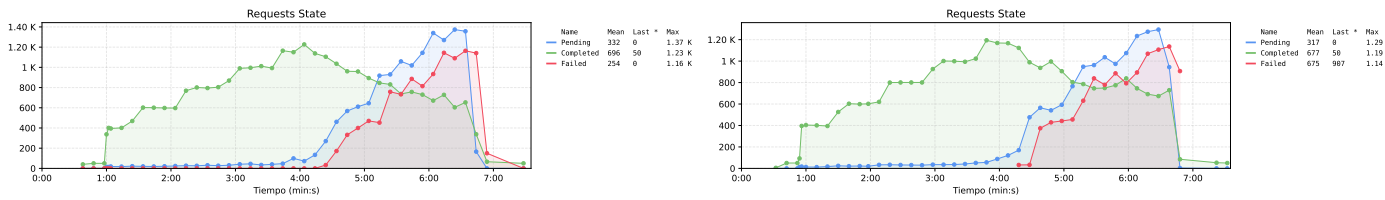


Figura 16.

TODO: completar

5.3.2. Latencia

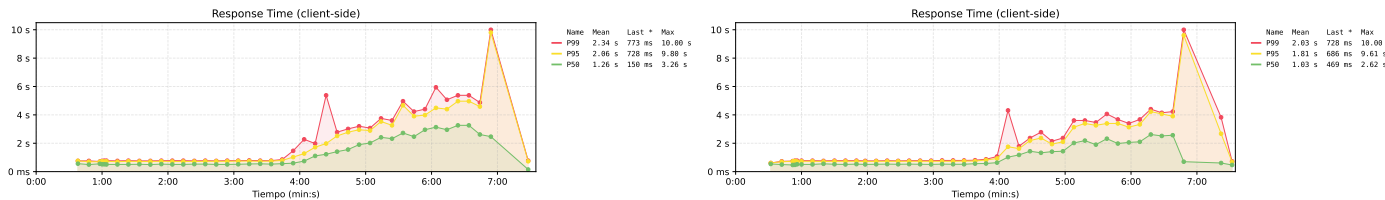


Figura 17.

TODO: completar

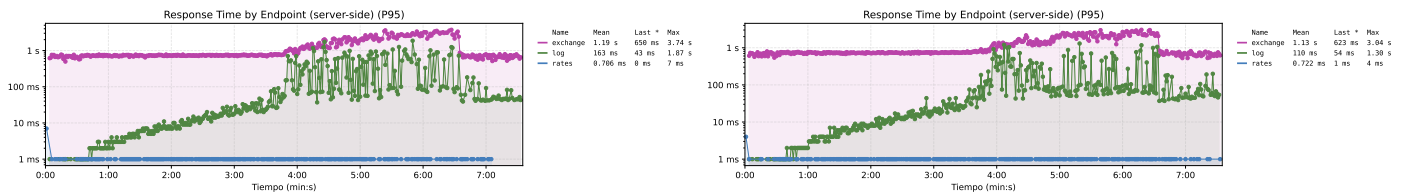


Figura 18.

TODO: completar

5.3.3. Uso de recursos

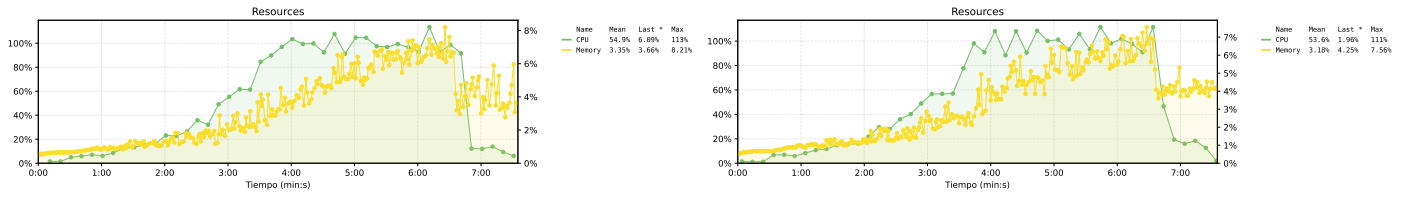


Figura 19.

TODO: completar

5.3.4. Volumen operado por moneda

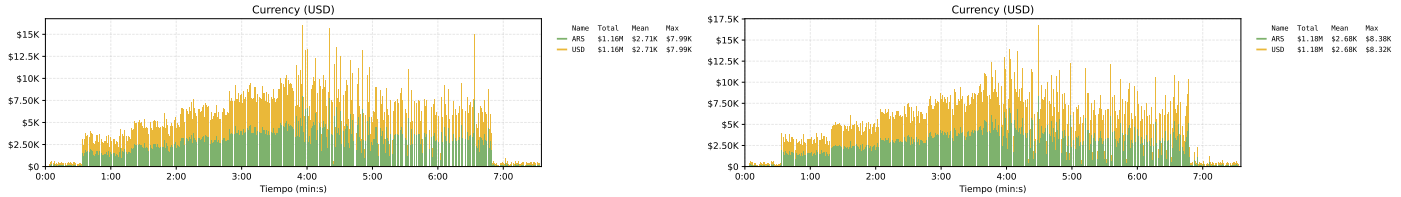


Figura 20.

TODO: completar

5.3.5. Volumen neto operado

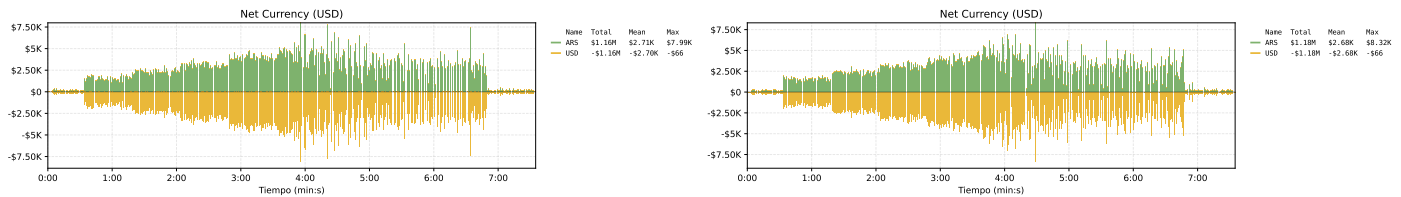


Figura 21.

TODO: completar

5.4. Pruebas de resistencia

Se pueden encontrar los resultados completos en el Apéndice.

5.4.1. Disponibilidad

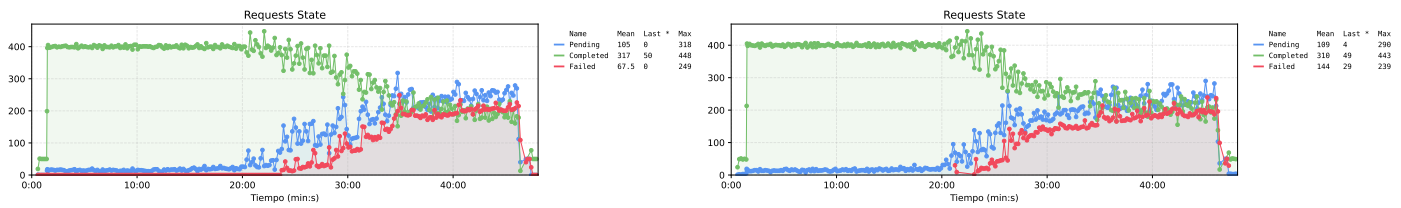


Figura 22.

TODO: completar

5.4.2. Latencia

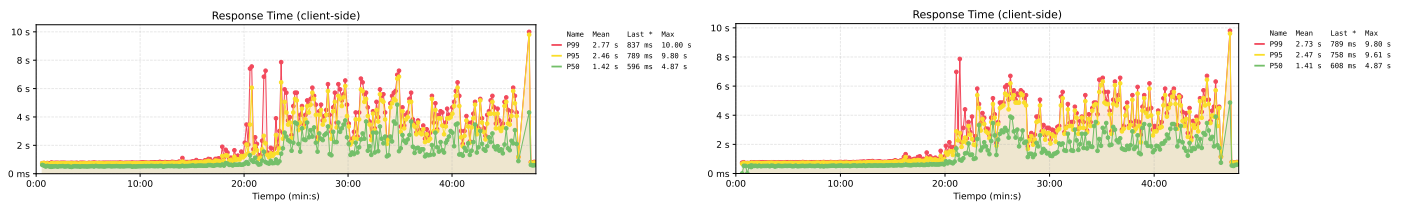


Figura 23.

TODO: completar

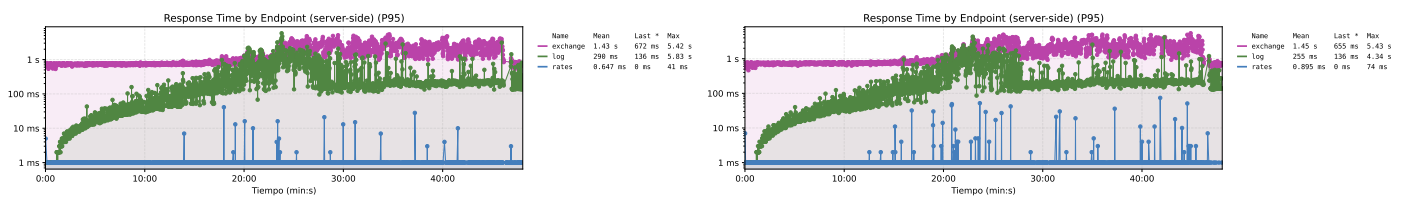


Figura 24.

TODO: completar

5.4.3. Uso de recursos

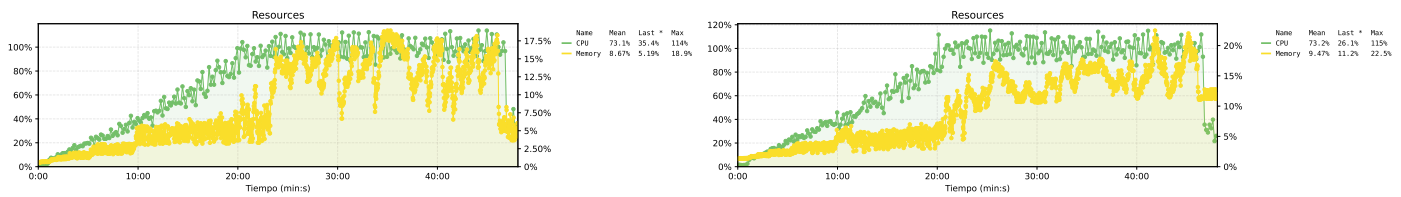
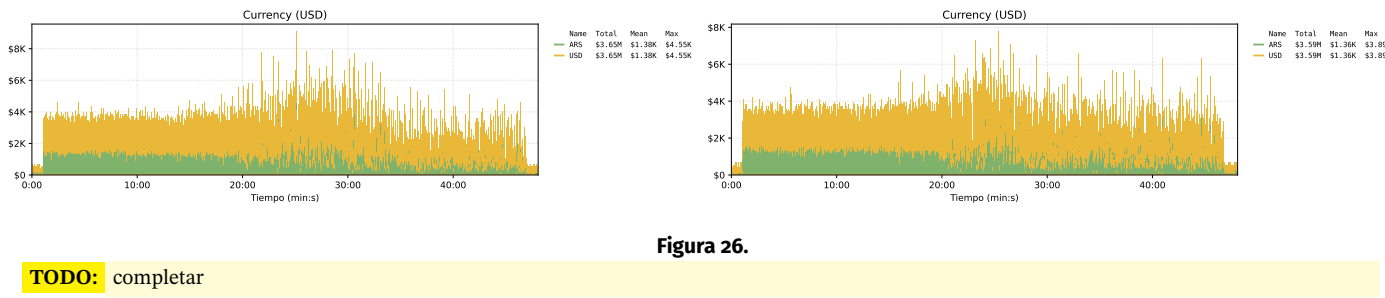


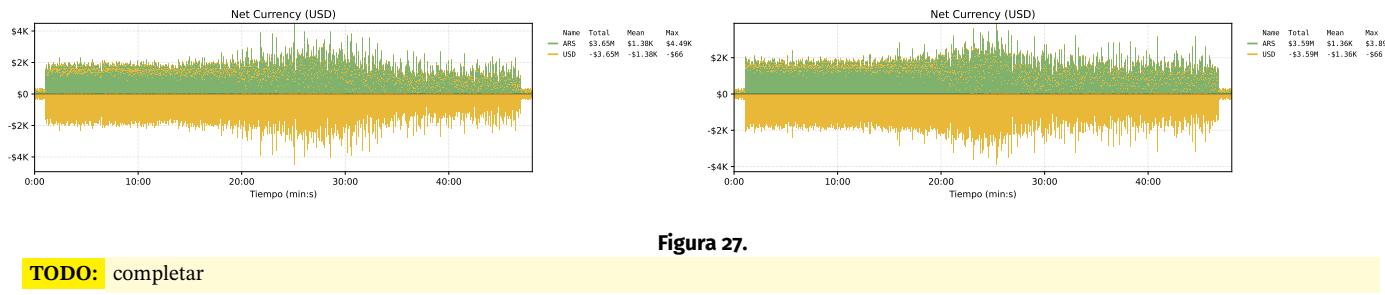
Figura 25.

TODO: completar

5.4.4. Volumen operado por moneda



5.4.5. Volumen neto operado

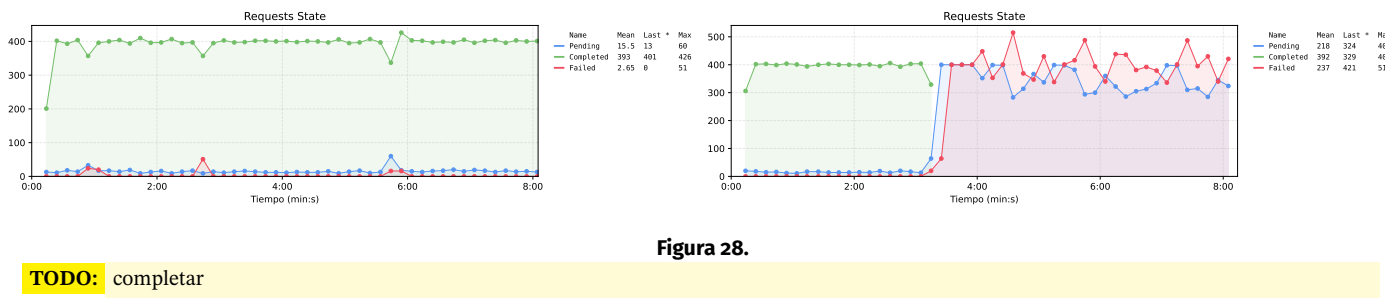


5.5. Pruebas de caos

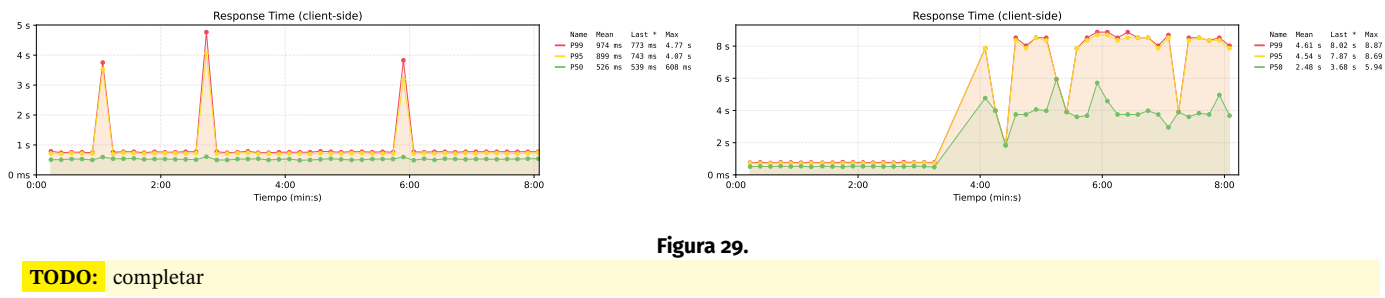
TODO: describir el escenario

Se pueden encontrar los resultados completos en el [Apendice](#).

5.5.1. Disponibilidad



5.5.2. Latencia



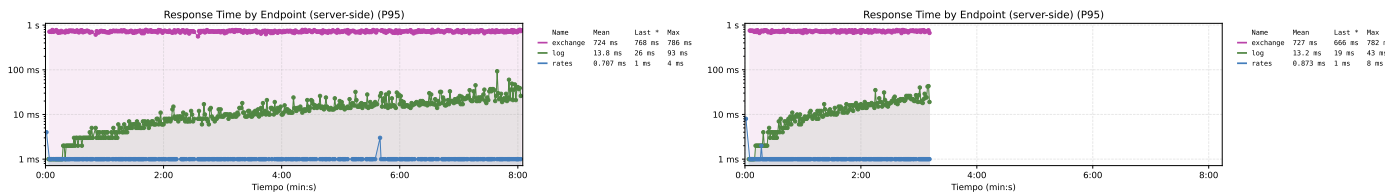


Figura 30.

TODO: completar

5.5.3. Uso de recursos

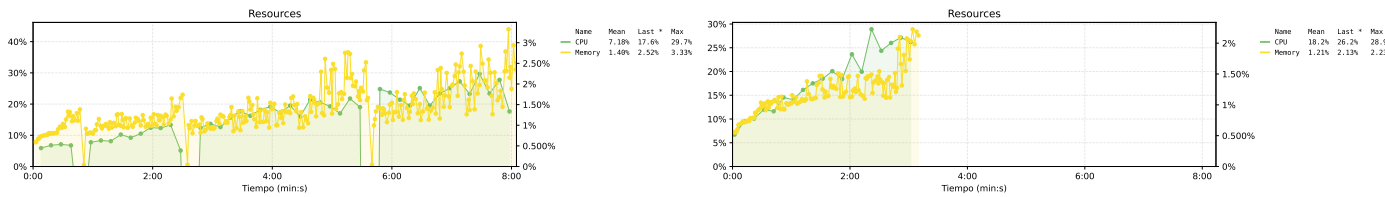


Figura 31.

TODO: completar

5.5.4. Volumen operado por moneda

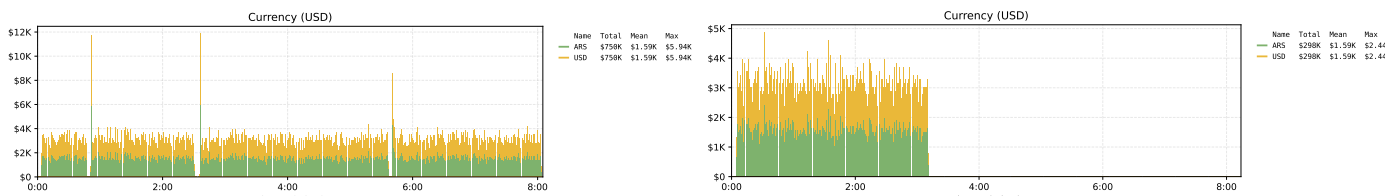


Figura 32.

TODO: completar

5.5.5. Volumen neto operado

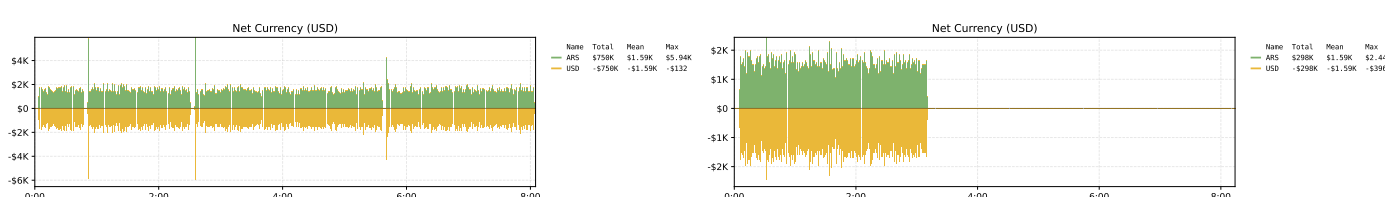


Figura 33.

TODO: completar

5.6. Resultados y conclusiones

6. USO DE HERRAMIENTAS DE IA

...

A. APENDICE

A.1. Resultados completos del escenario de carga sobre el sistema base

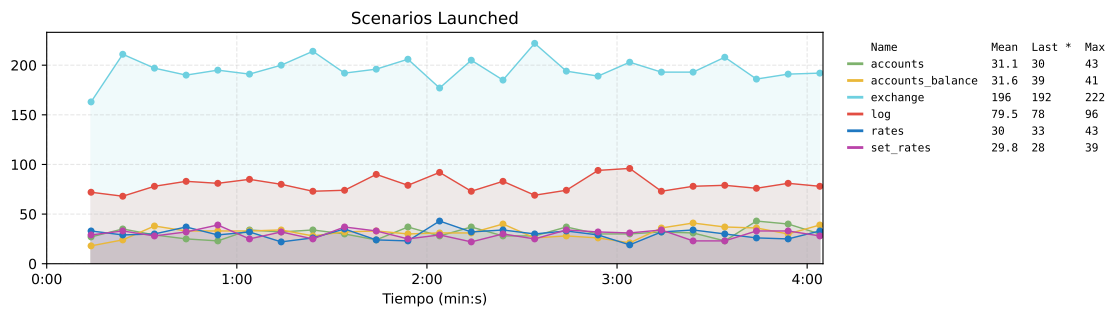


Figura 34. Escenarios del sistema base.

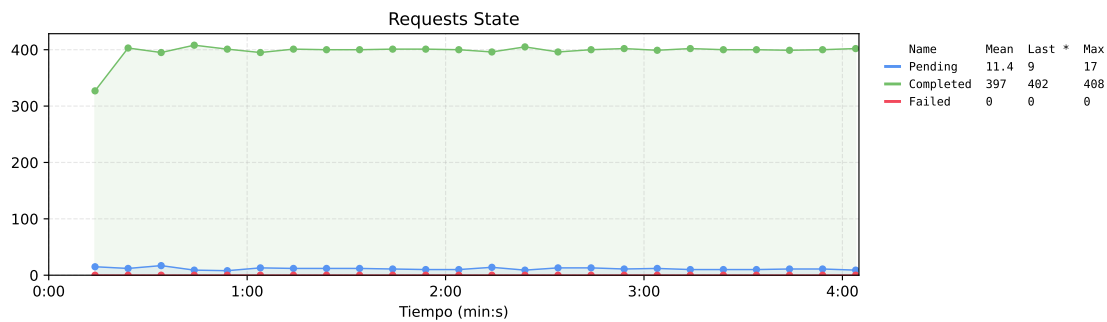


Figura 35. Estado de las requests del sistema base.

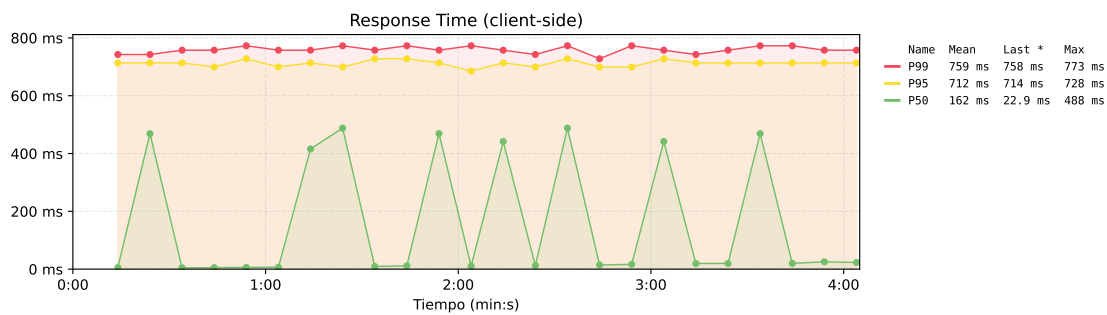


Figura 36. Tiempo de respuesta del sistema base.

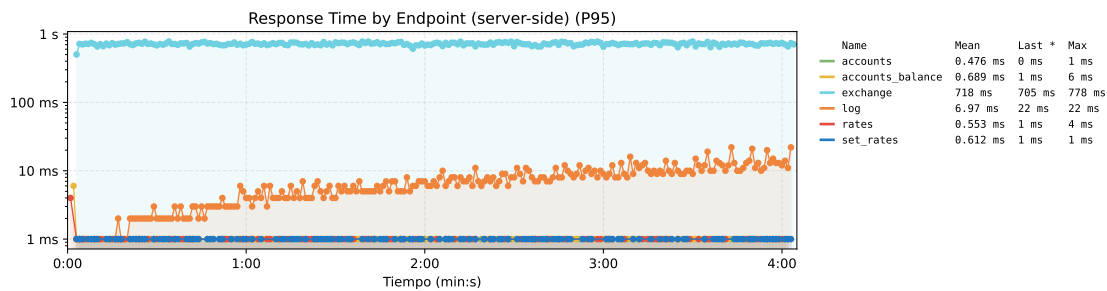


Figura 37. Tiempo de respuesta por endpoint del sistema base. El eje tiene una escala logarítmica.

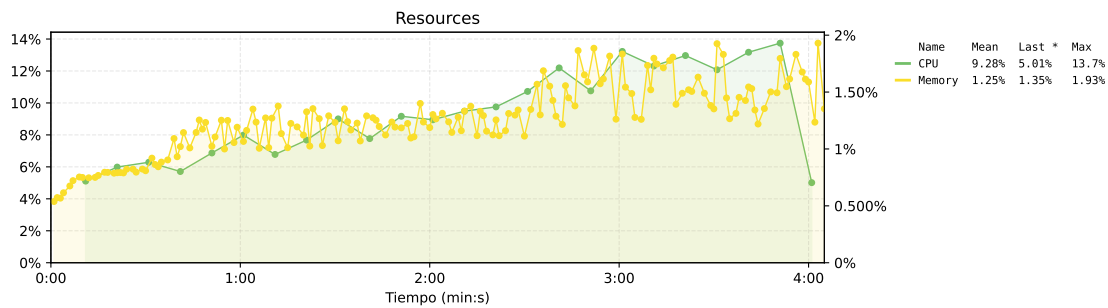


Figura 38. Estado de los recursos del sistema base. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

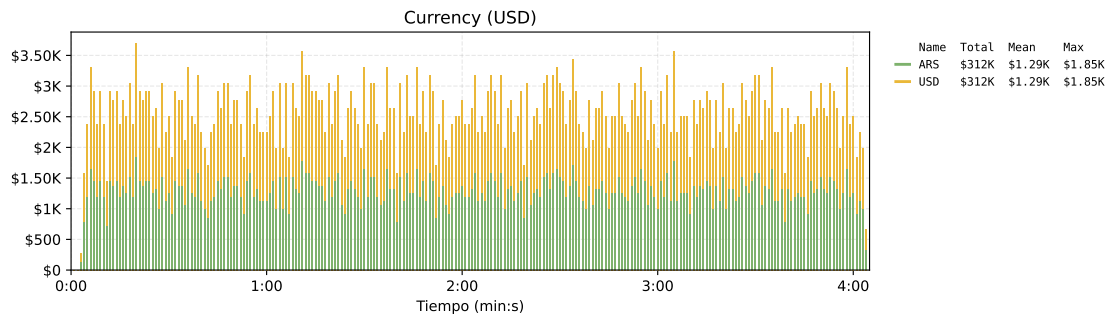


Figura 39. Volumen operado en cada moneda en el sistema base.

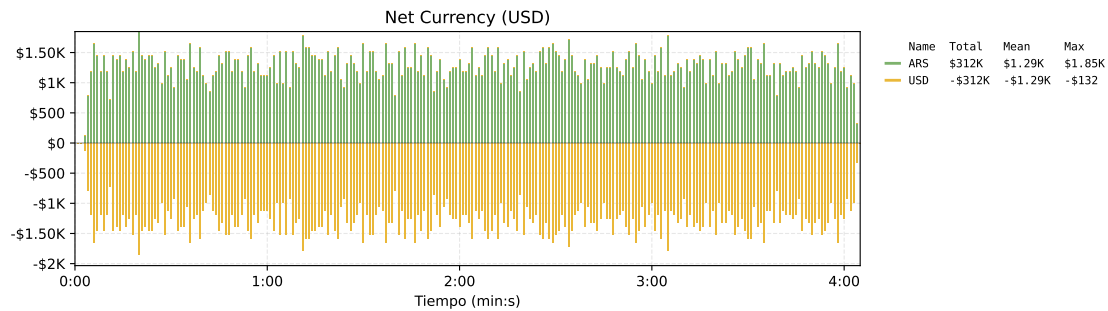


Figura 40. Neto operado en cada moneda en el sistema base.

A.2. Resultados completos del escenario de estrés sobre el sistema base

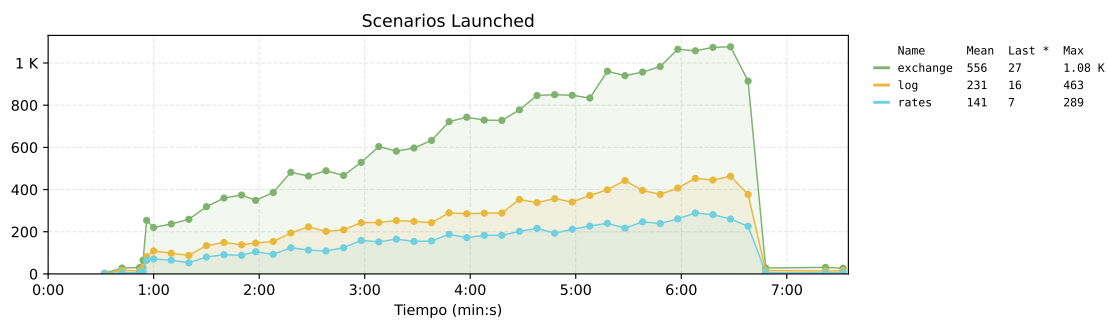


Figura 41. Escenarios del sistema base.

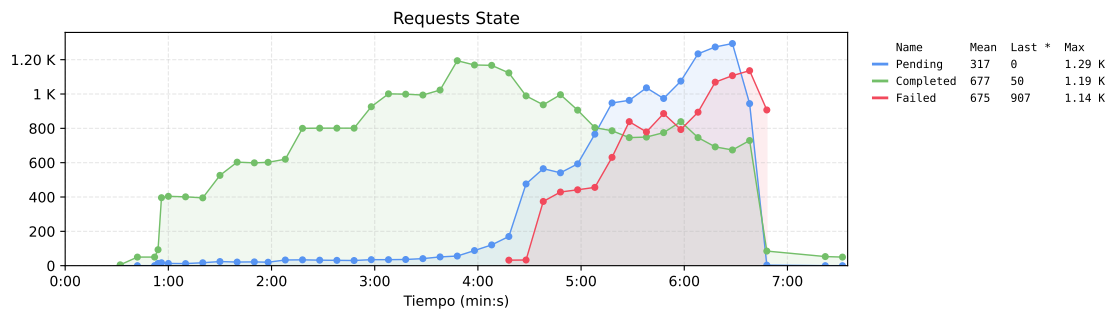


Figura 42. Estado de las requests del sistema base.

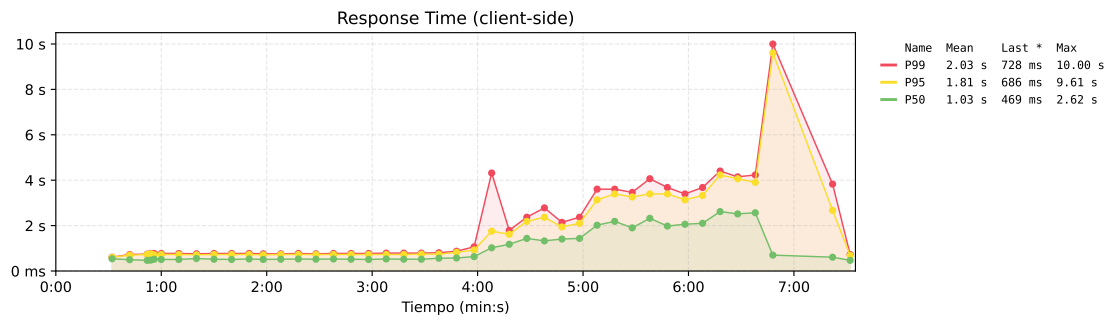


Figura 43. Tiempo de respuesta del sistema base.

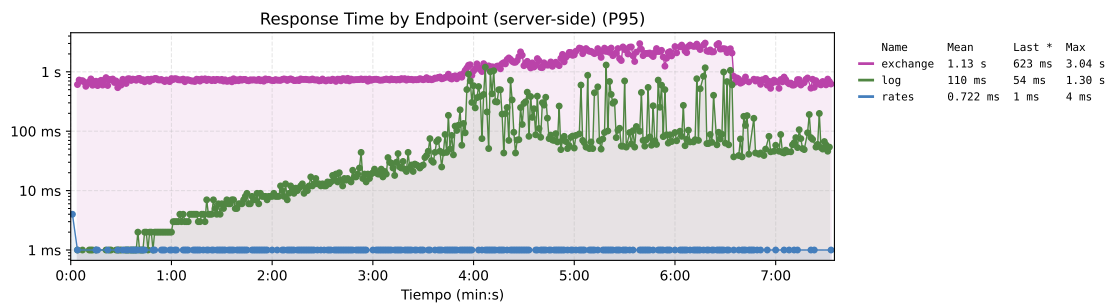


Figura 44. Tiempo de respuesta por endpoint del sistema base. El eje tiene una escala logarítmica.

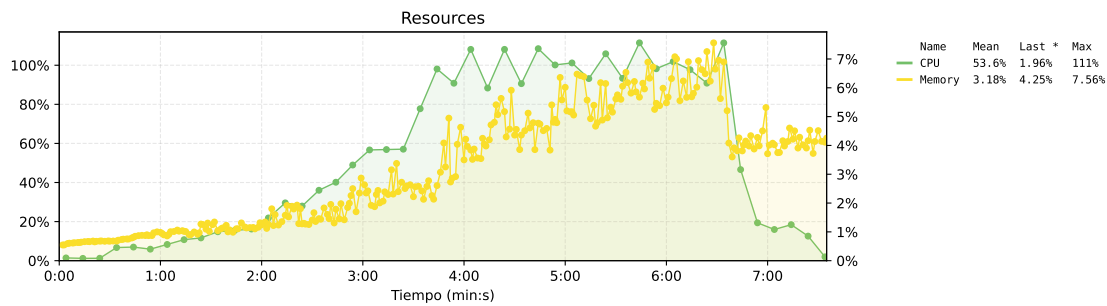


Figura 45. Estado de los recursos del sistema base. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

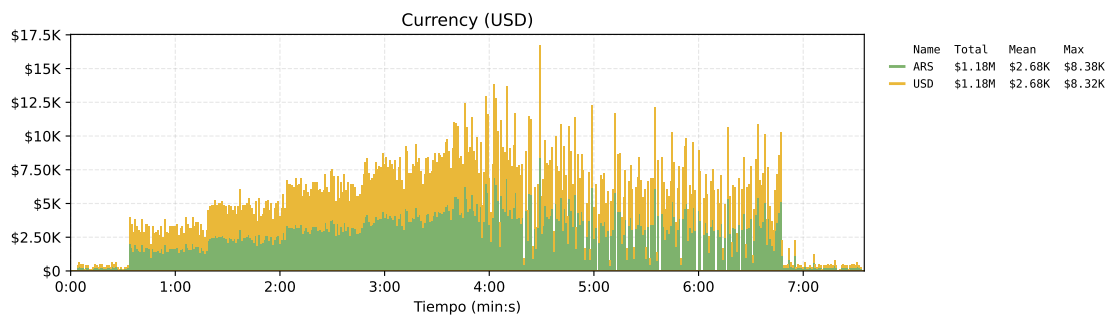


Figura 46. Volumen operado en cada moneda en el sistema base.

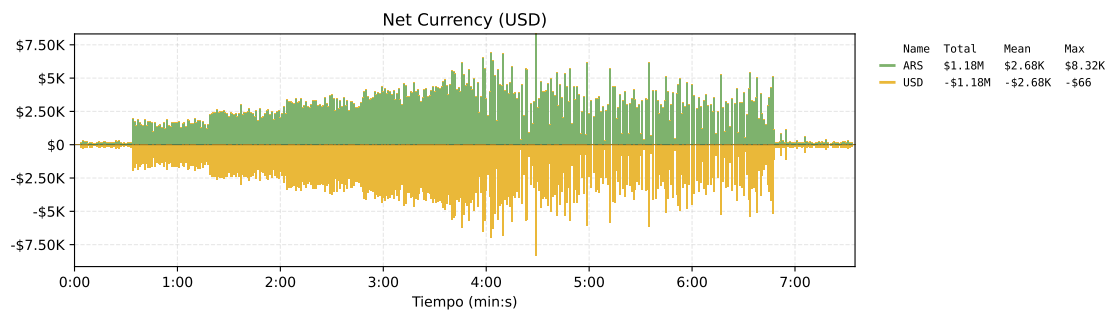


Figura 47. Neto operado en cada moneda en el sistema base.

A.3. Resultados completos del escenario de resistencia sobre el sistema base

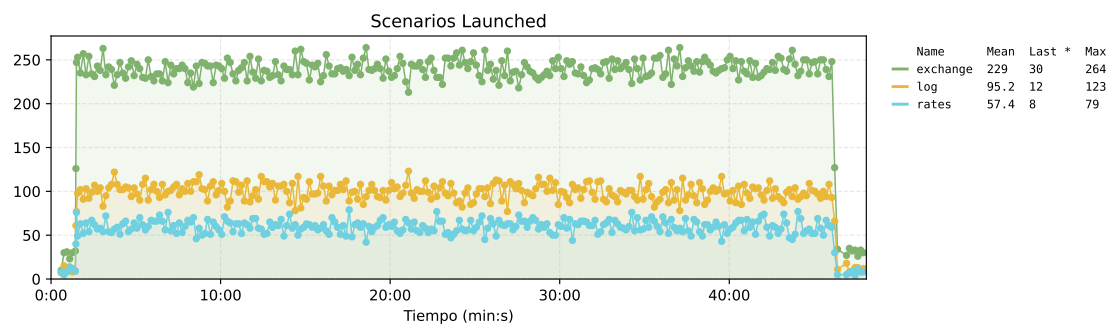


Figura 48. Escenarios del sistema base.

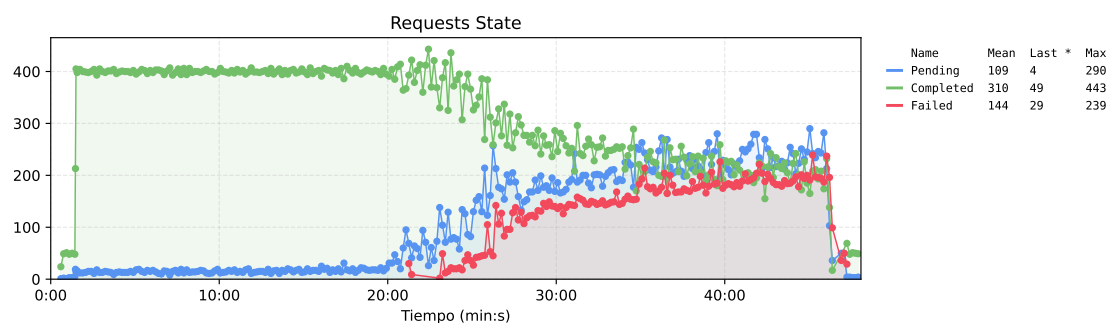


Figura 49. Estado de las requests del sistema base.



Figura 50. Tiempo de respuesta del sistema base.

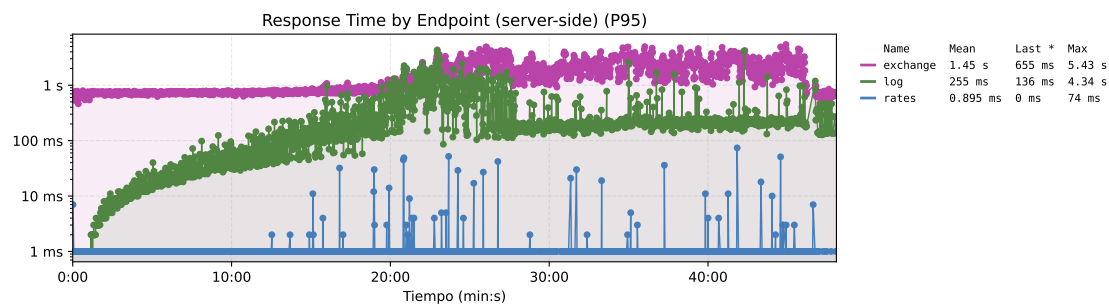


Figura 51. Tiempo de respuesta por endpoint del sistema base. El eje tiene una escala logarítmica.

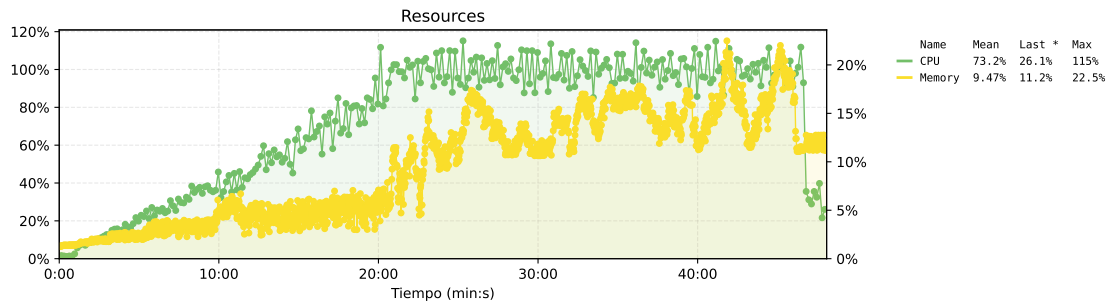


Figura 52. Estado de los recursos del sistema base. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

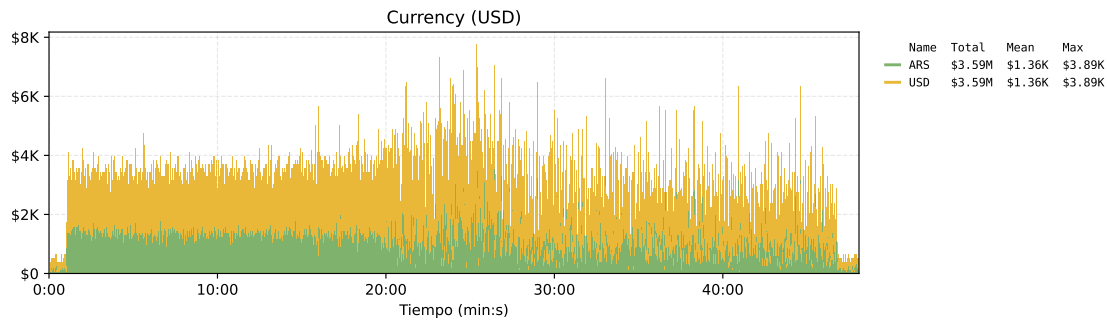


Figura 53. Volumen operado en cada moneda en el sistema base.

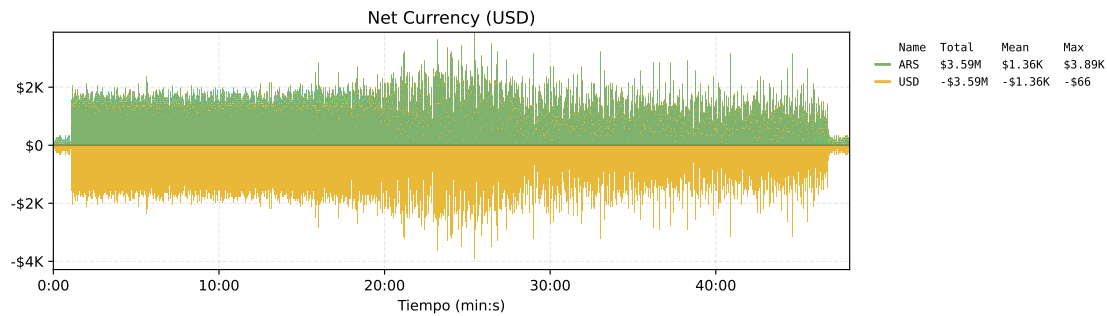


Figura 54. Neto operado en cada moneda en el sistema base.

A.4. Resultados completos del escenario de caos sobre el sistema base

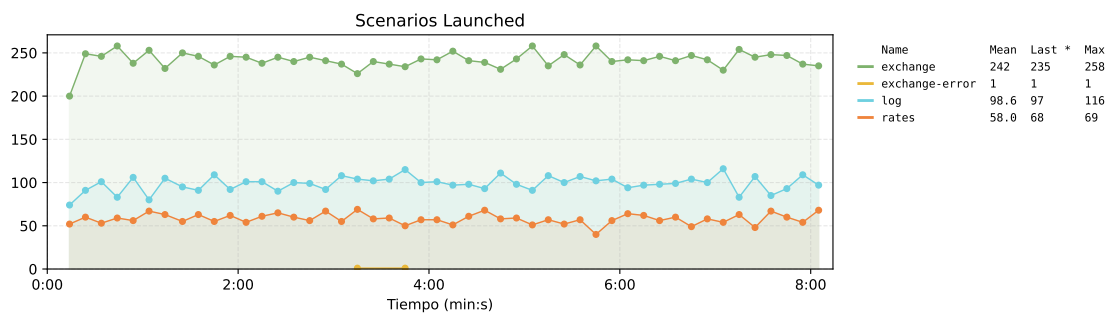


Figura 55. Escenarios del sistema base.

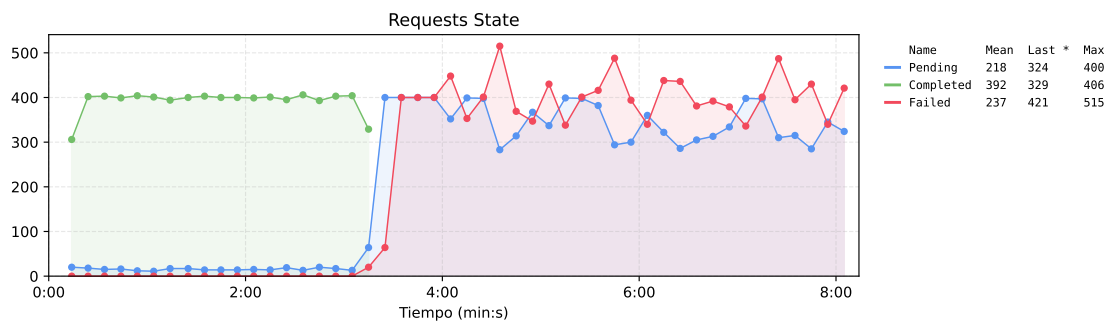


Figura 56. Estado de las requests del sistema base.

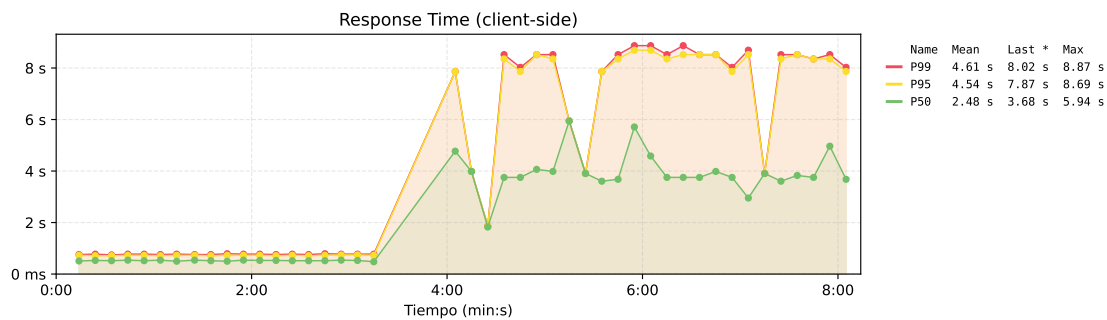


Figura 57. Tiempo de respuesta del sistema base.

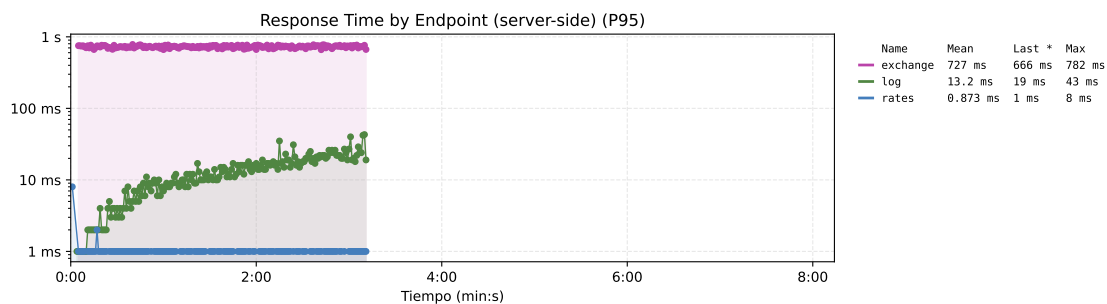


Figura 58. Tiempo de respuesta por endpoint del sistema base. El eje tiene una escala logaritmica.

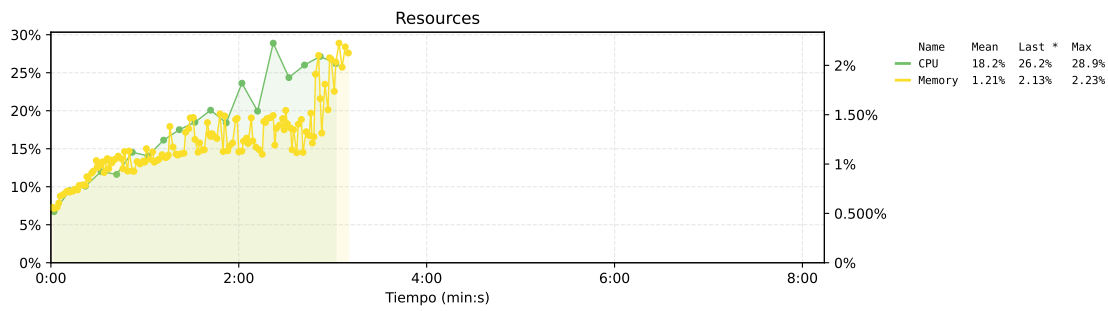


Figura 59. Estado de los recursos del sistema base. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

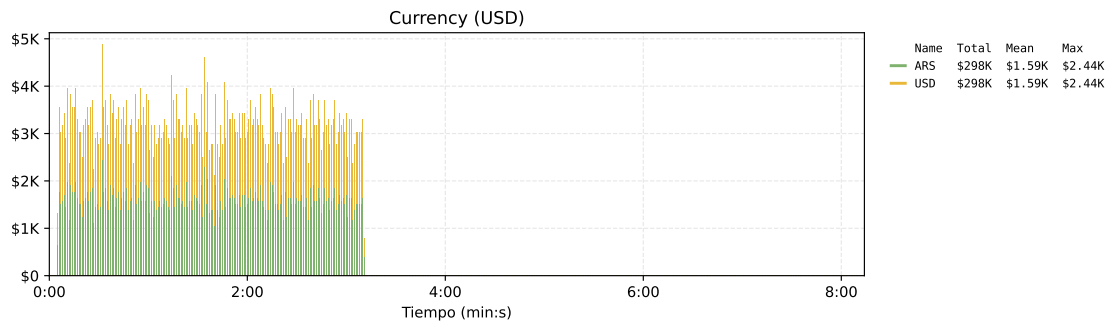


Figura 60. Volumen operado en cada moneda en el sistema base.

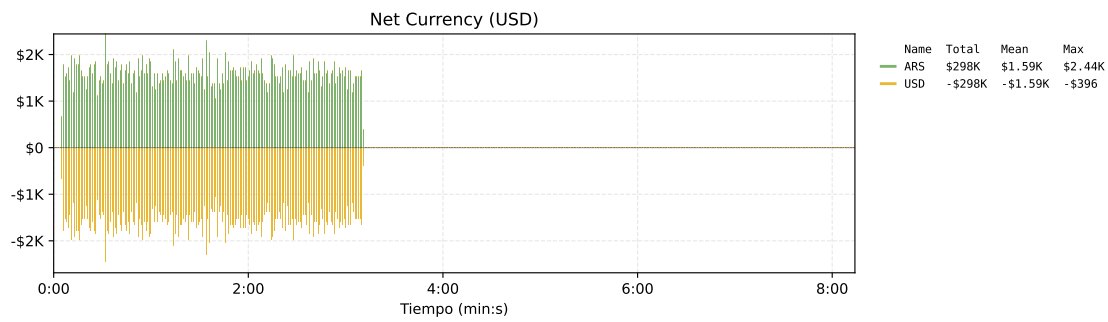


Figura 61. Neto operado en cada moneda en el sistema base.

A.5. Resultados completos del escenario de carga sobre la propuesta 1

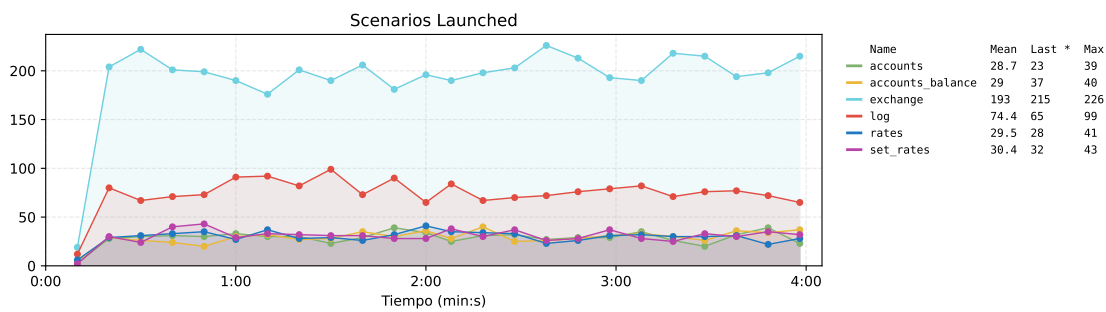


Figura 62. Escenarios de la propuesta 1.

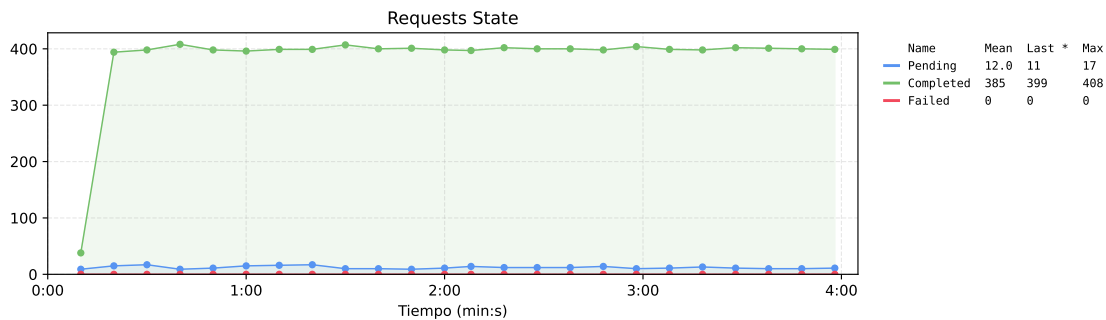


Figura 63. Estado de las requests de la propuesta 1.

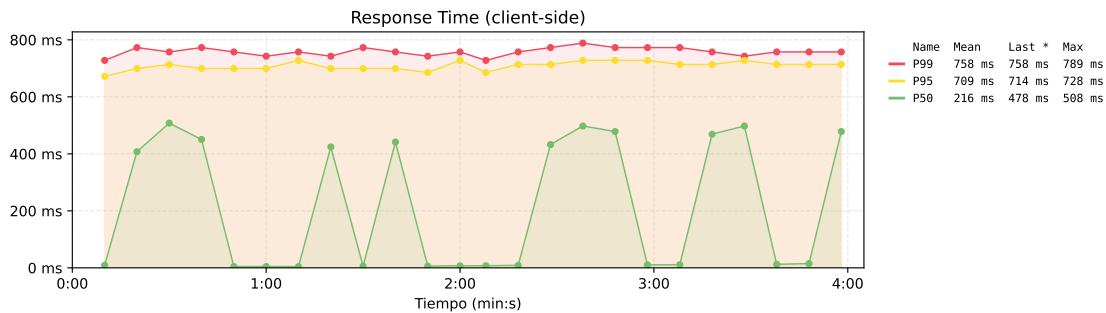


Figura 64. Tiempo de respuesta de la propuesta 1.

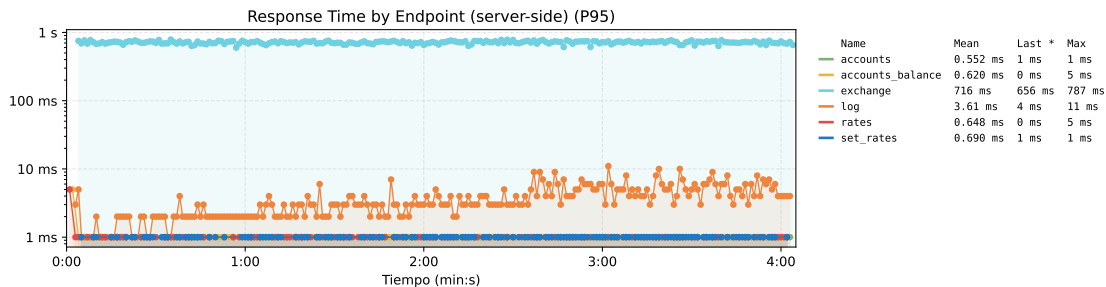


Figura 65. Tiempo de respuesta por endpoint de la propuesta 1. El eje tiene una escala logarítmica.

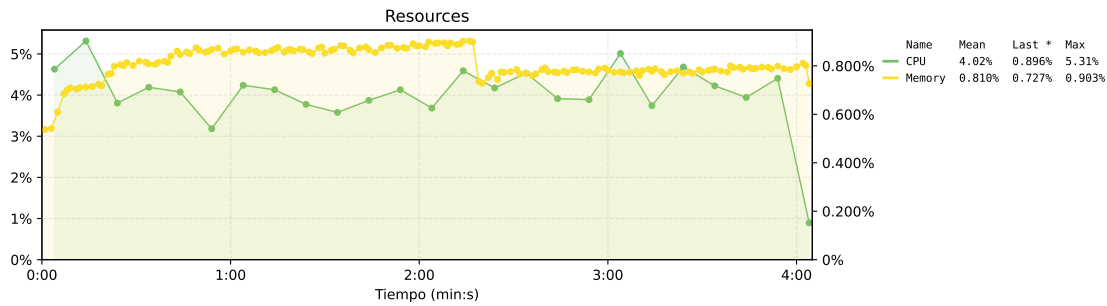


Figura 66. Estado de los recursos del nodo de exchange de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

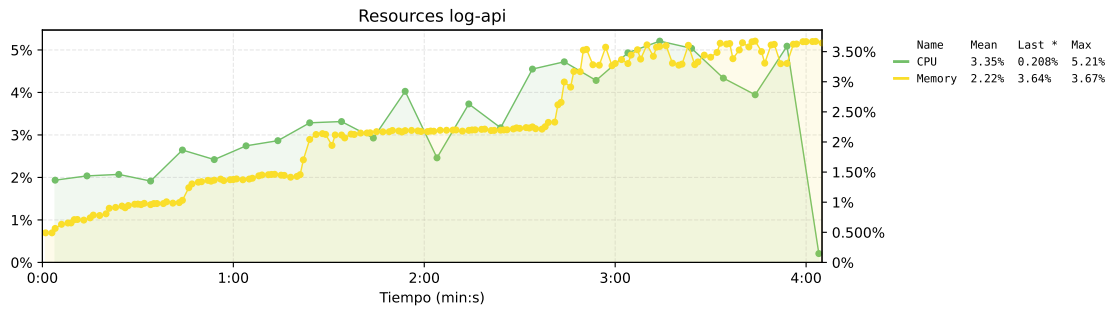


Figura 67. Estado de los recursos del nodo de logs de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

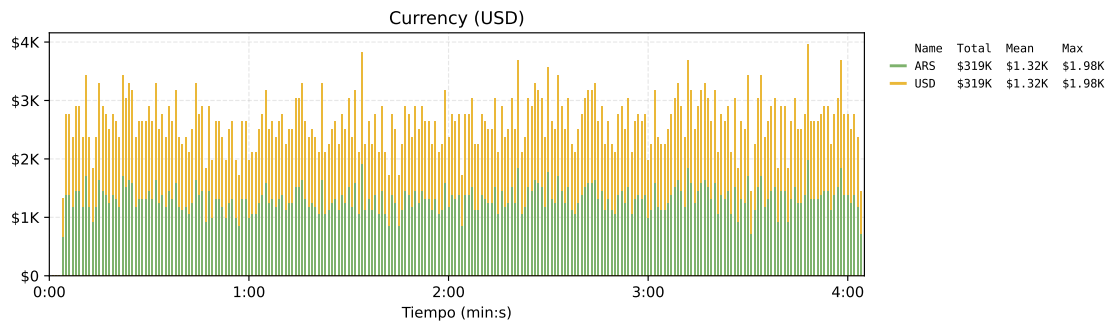


Figura 68. Volumen operado en cada moneda en la propuesta 1.

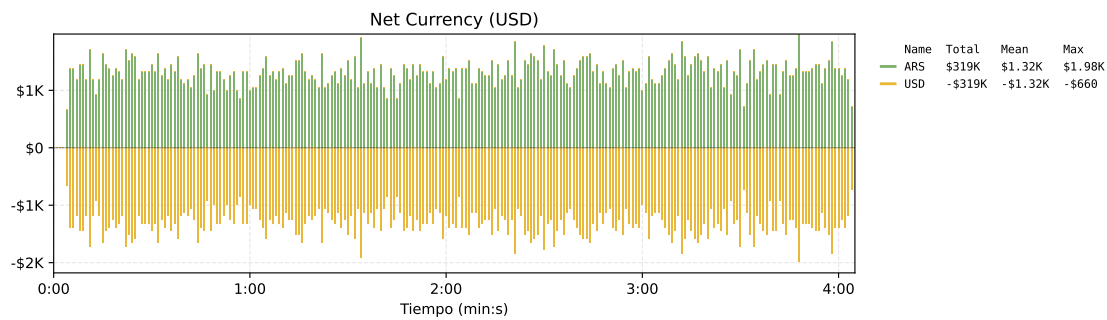


Figura 69. Neto operado en cada moneda en la propuesta 1.

A.6. Resultados completos del escenario de estrés sobre la propuesta 1

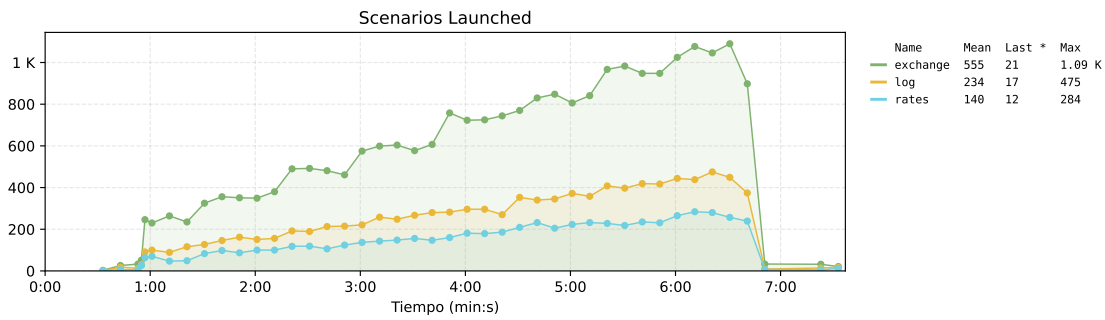


Figura 70. Escenarios de la propuesta 1.

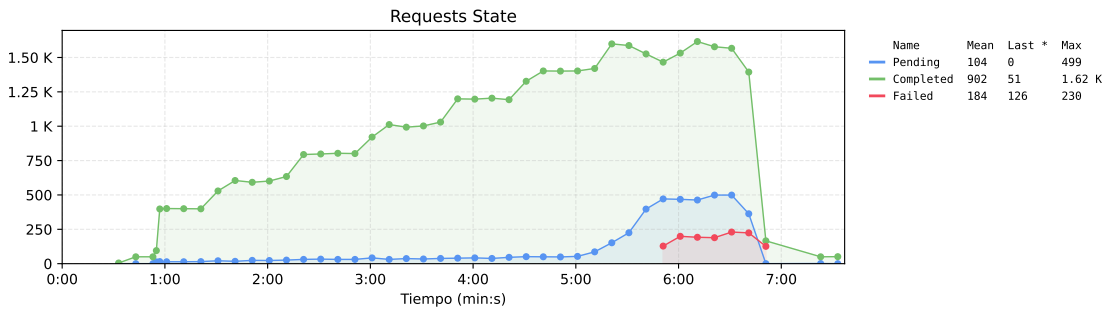


Figura 71. Estado de las requests de la propuesta 1.

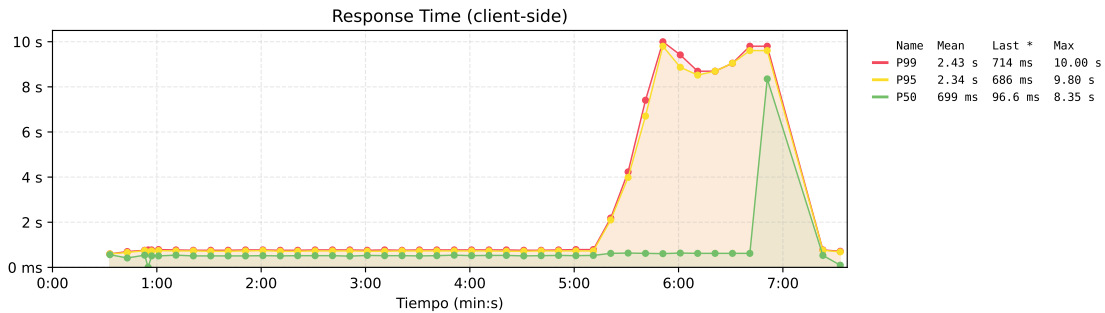


Figura 72. Tiempo de respuesta de la propuesta 1.

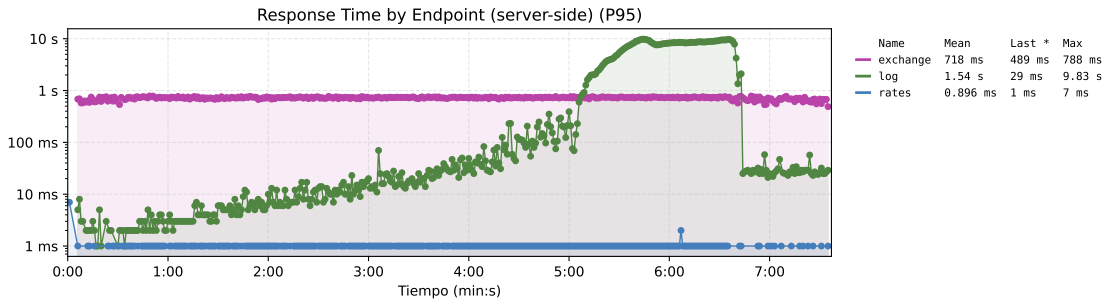


Figura 73. Tiempo de respuesta por endpoint de la propuesta 1. El eje tiene una escala logarítmica.

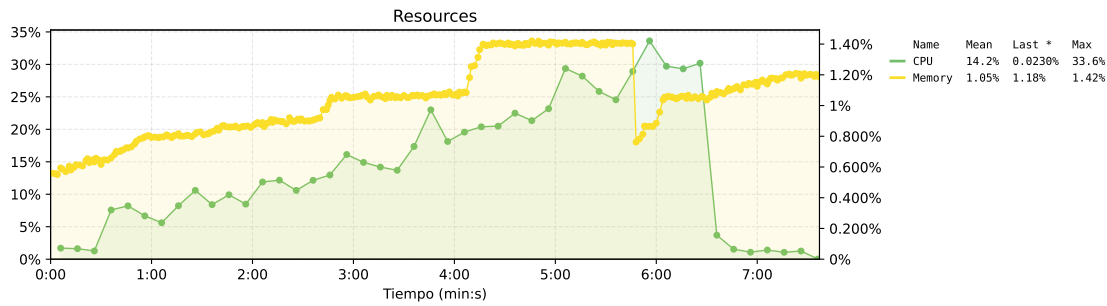


Figura 74. Estado de los recursos del nodo de exchange de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

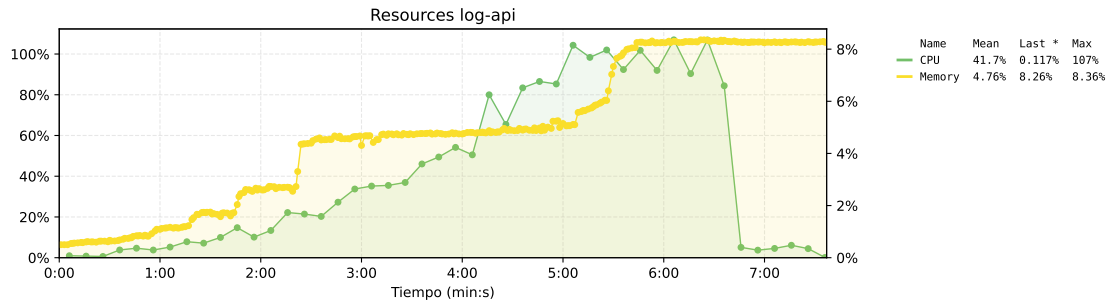


Figura 75. Estado de los recursos del nodo de logs de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

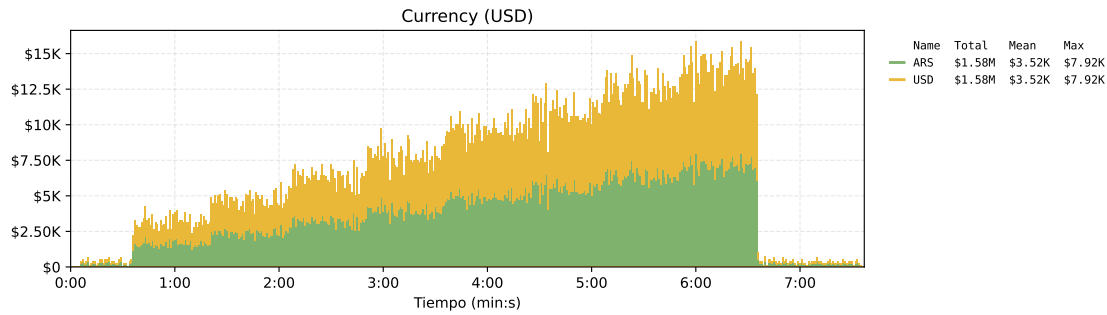


Figura 76. Volumen operado en cada moneda en la propuesta 1.

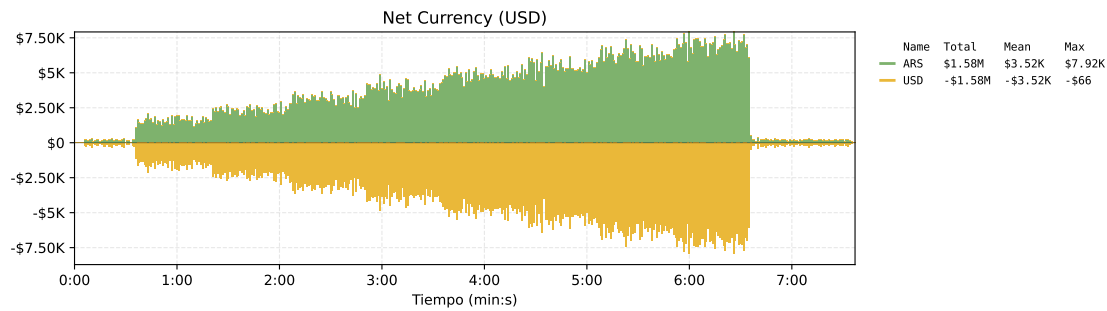


Figura 77. Neto operado en cada moneda en la propuesta 1.

A.7. Resultados completos del escenario de resistencia sobre la propuesta 1

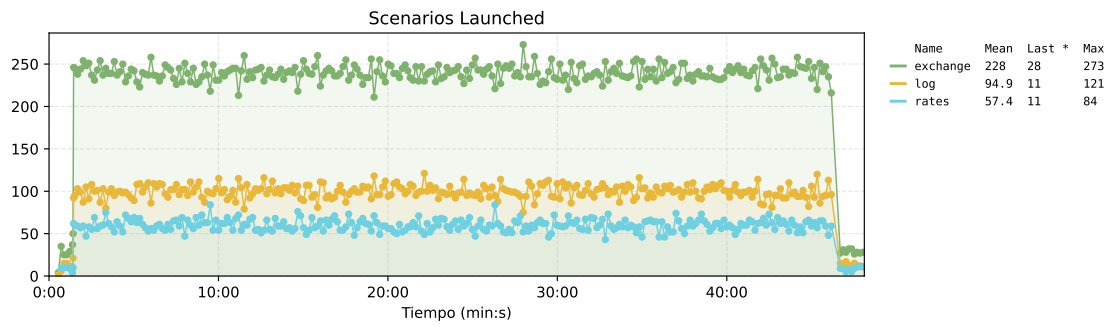


Figura 78. Escenarios de la propuesta 1.

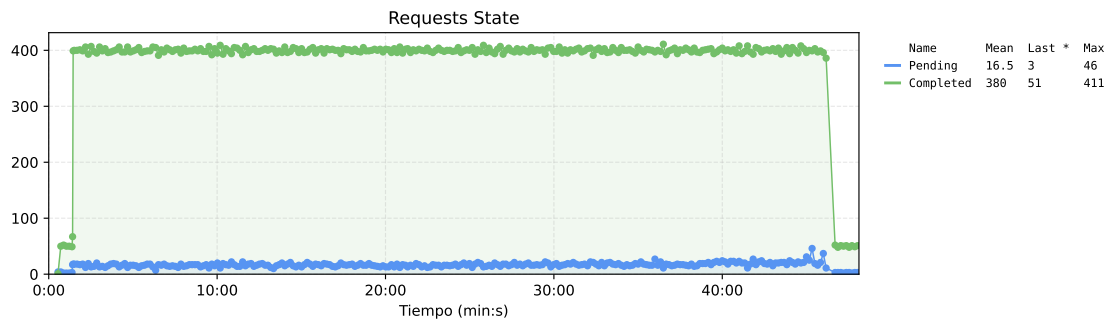


Figura 79. Estado de las requests de la propuesta 1.

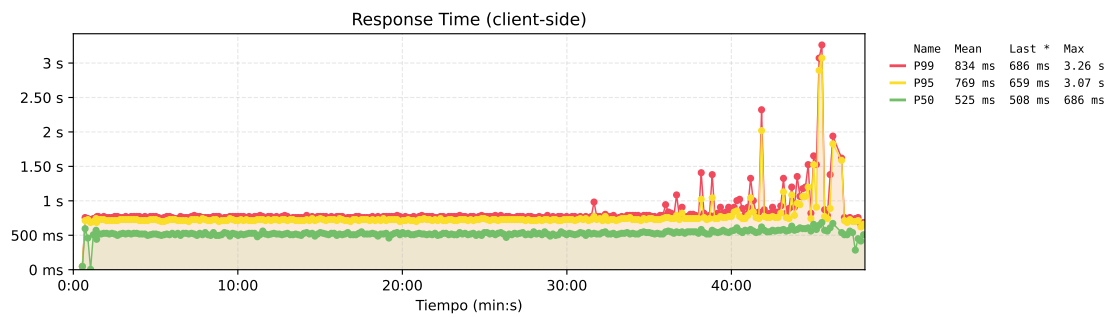


Figura 80. Tiempo de respuesta de la propuesta 1.

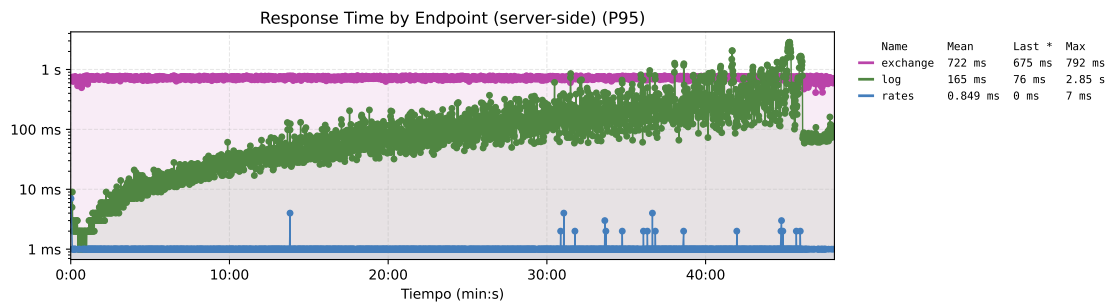


Figura 81. Tiempo de respuesta por endpoint de la propuesta 1. El eje tiene una escala logaritmica.

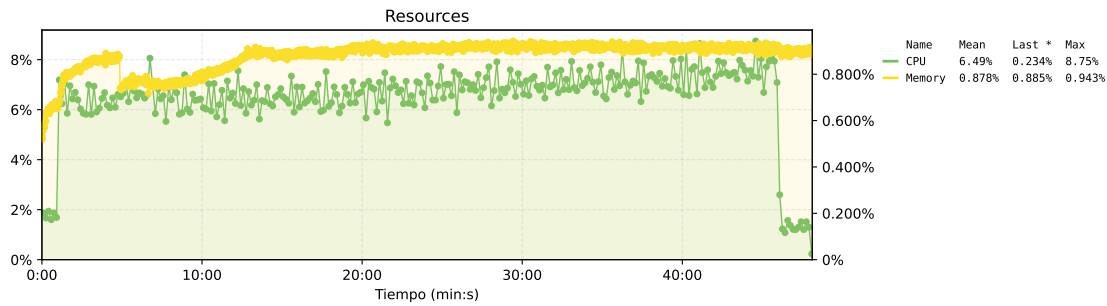


Figura 82. Estado de los recursos del nodo de exchange de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

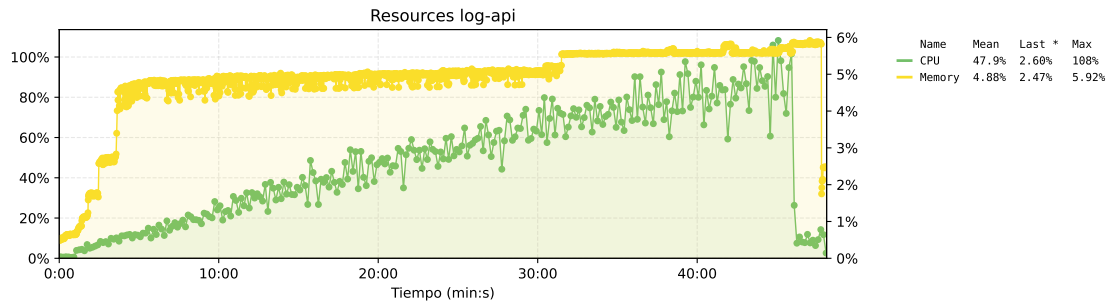


Figura 83. Estado de los recursos del nodo de logs de la propuesta 1. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

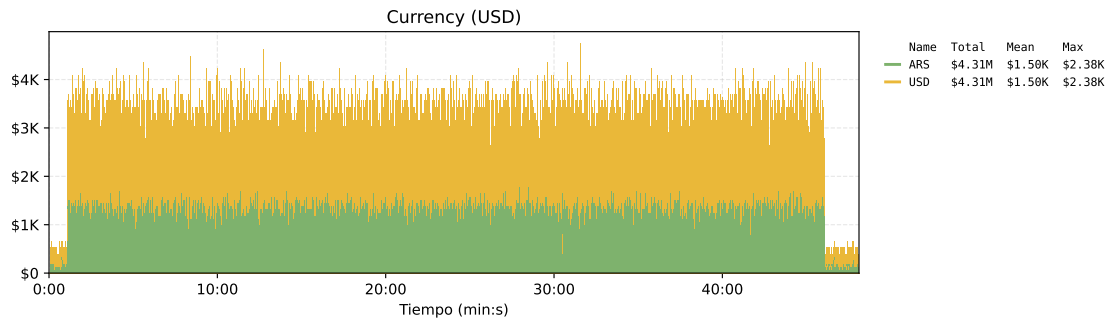


Figura 84. Volumen operado en cada moneda en la propuesta 1.

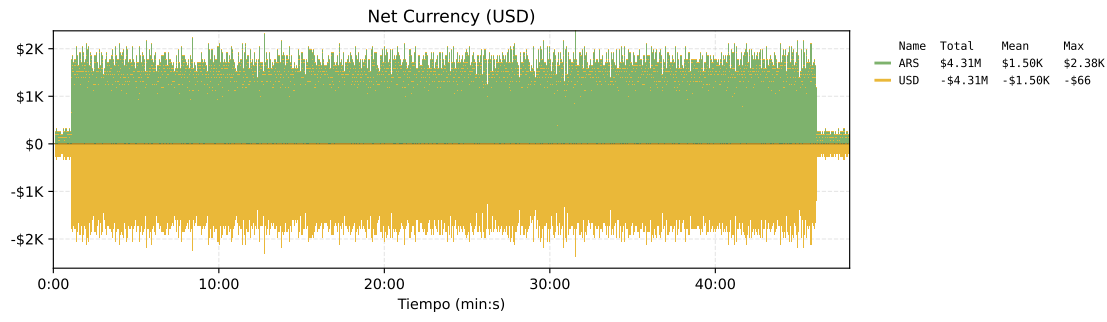


Figura 85. Neto operado en cada moneda en la propuesta 1.

A.8. Resultados completos del escenario de carga sobre la propuesta 2

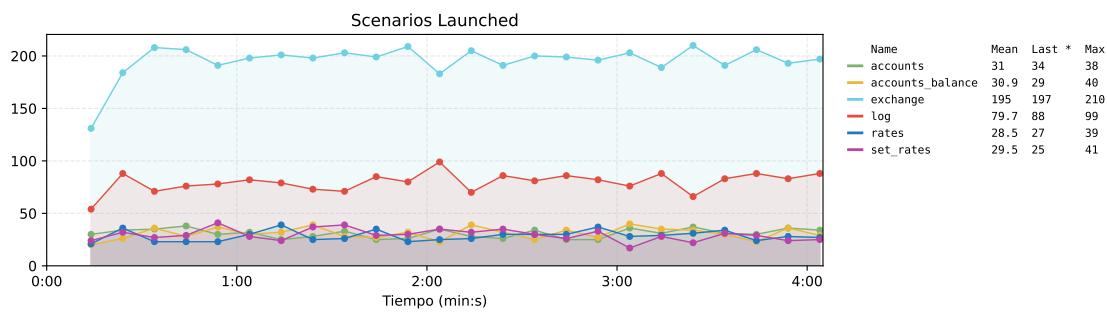


Figura 86. Escenarios de la propuesta 2.

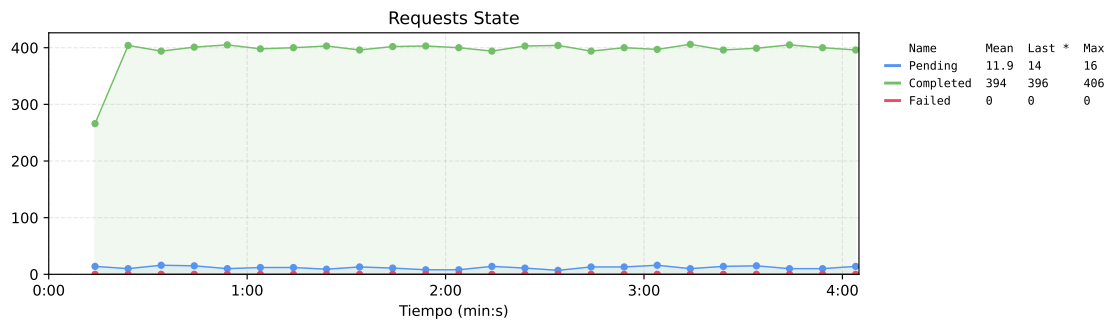


Figura 87. Estado de las requests de la propuesta 2.

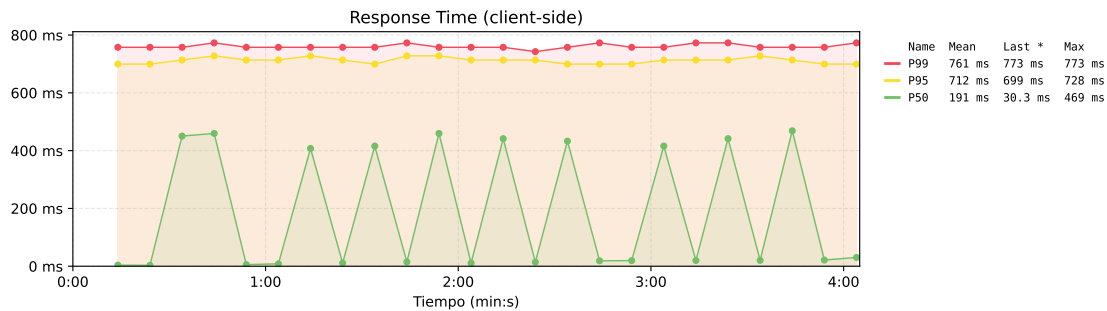


Figura 88. Tiempo de respuesta de la propuesta 2.

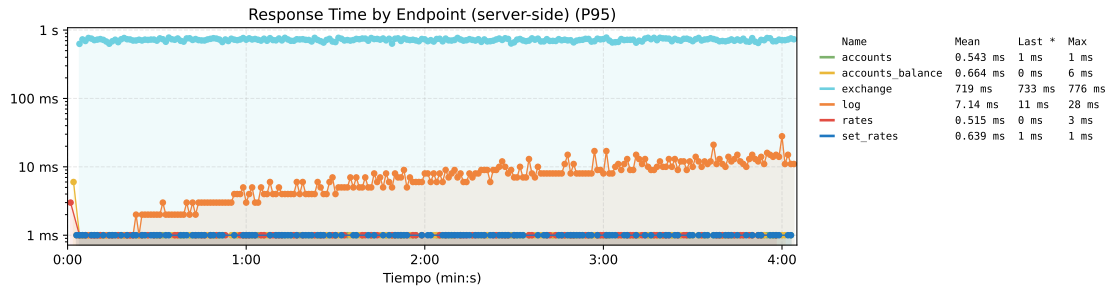


Figura 89. Tiempo de respuesta por endpoint de la propuesta 2. El eje tiene una escala logaritmica.

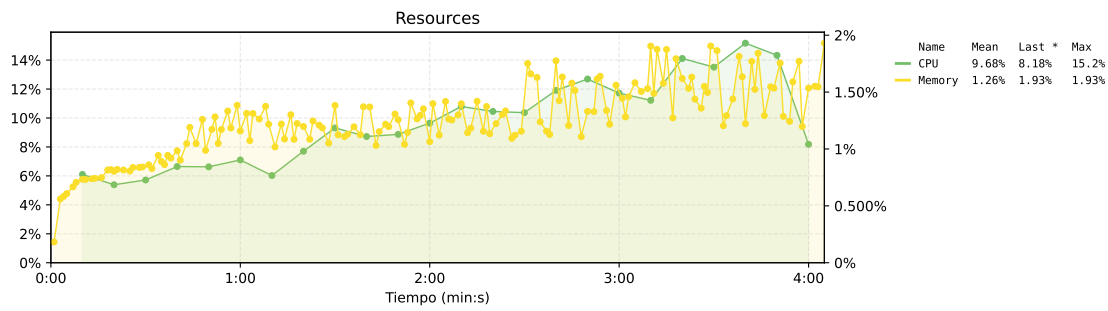


Figura 90. Estado de los recursos de la propuesta 2. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

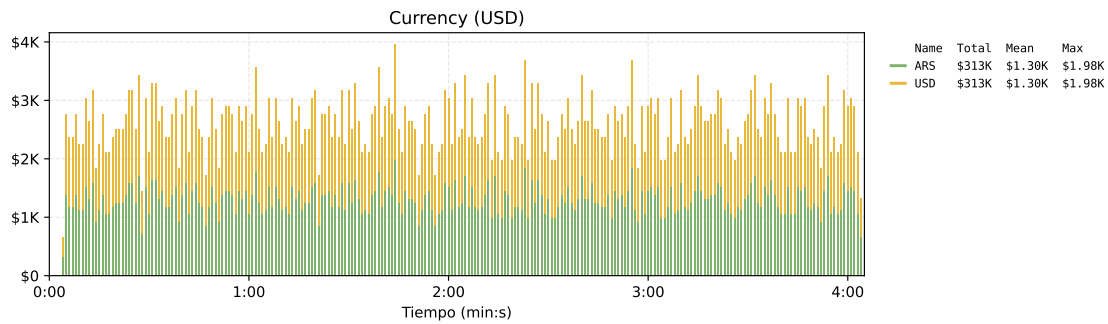


Figura 91. Volumen operado en cada moneda en la propuesta 2.

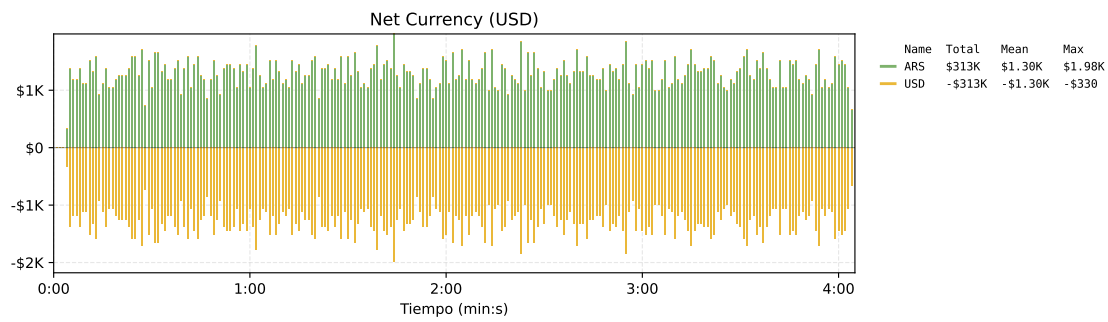


Figura 92. Neto operado en cada moneda en la propuesta 2.

A.9. Resultados completos del escenario de estrés sobre la propuesta 2

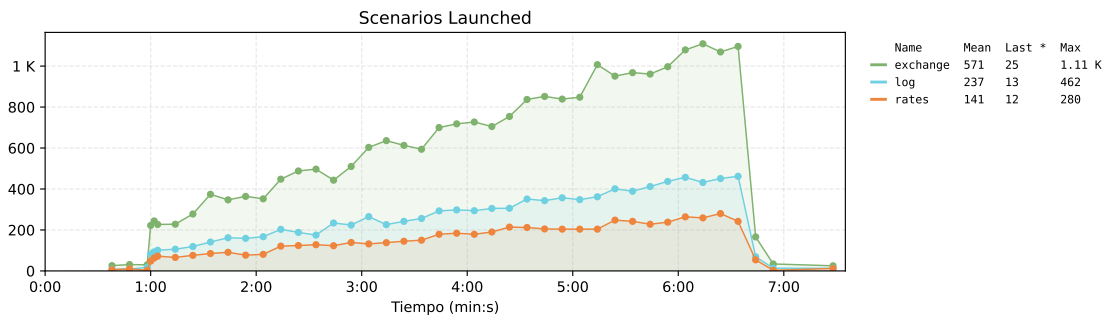


Figura 93. Escenarios de la propuesta 2.

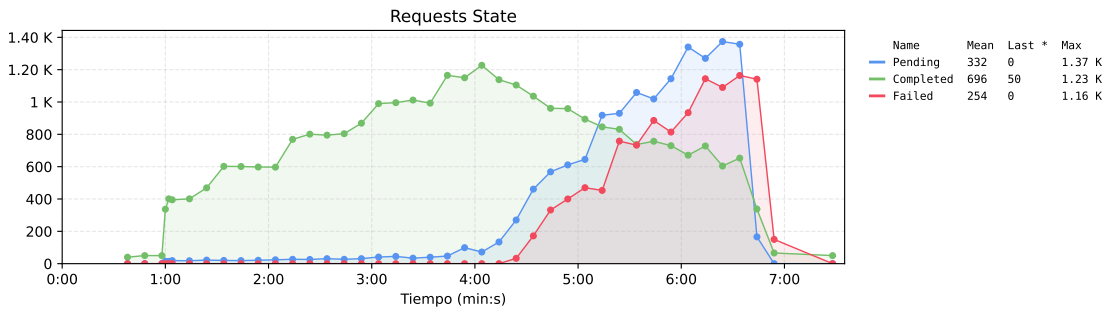


Figura 94. Estado de las requests de la propuesta 2.

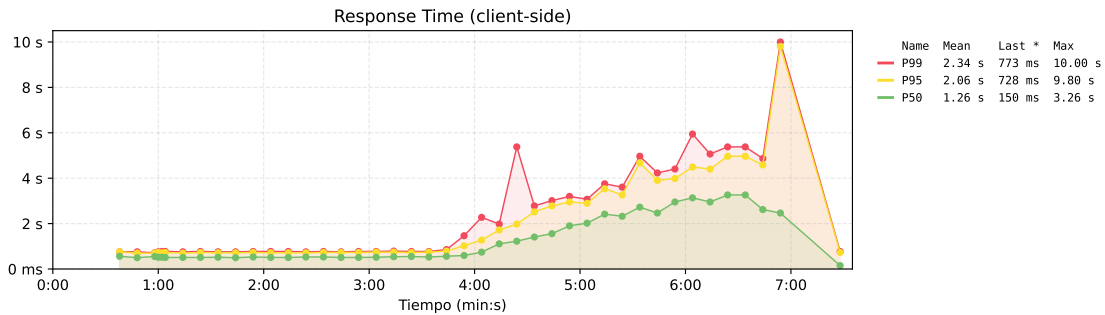


Figura 95. Tiempo de respuesta de la propuesta 2.

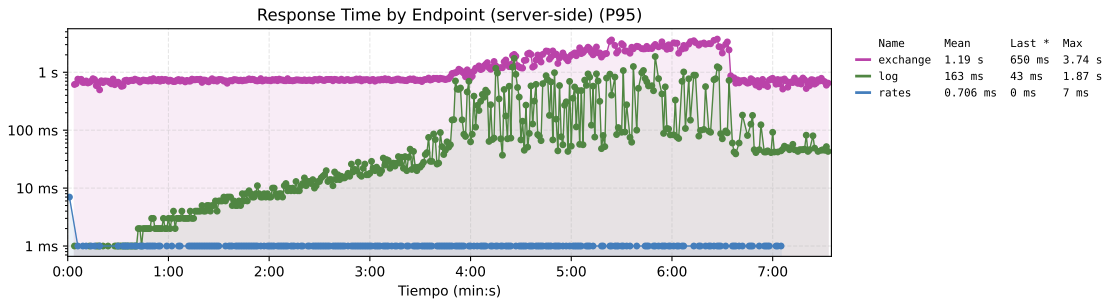


Figura 96. Tiempo de respuesta por endpoint de la propuesta 2. El eje tiene una escala logarítmica.

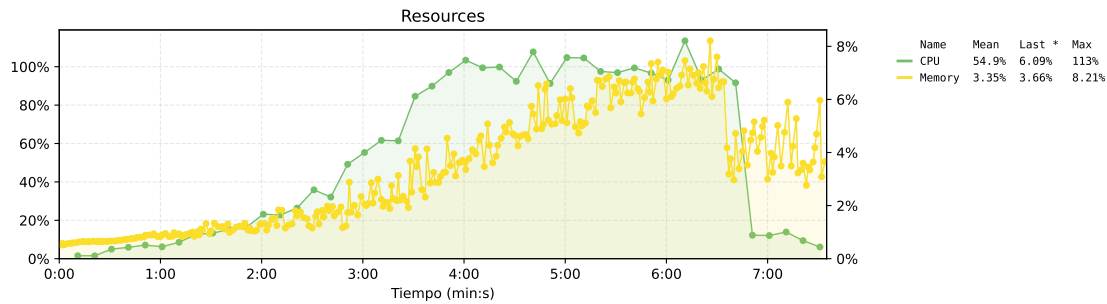


Figura 97. Estado de los recursos de la propuesta 2. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

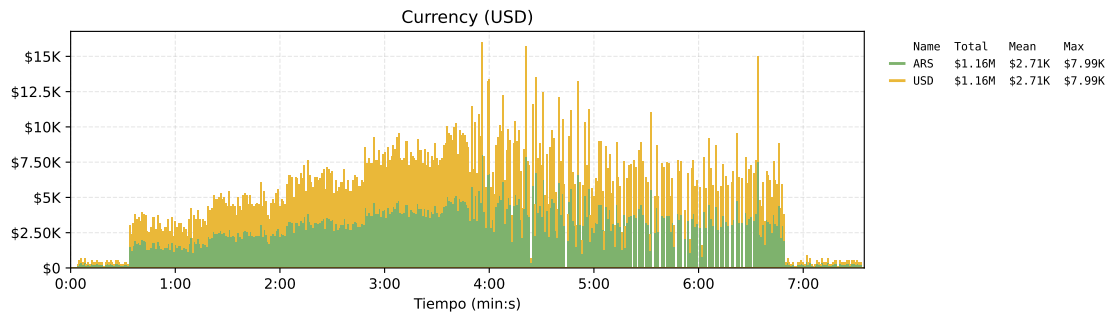


Figura 98. Volumen operado en cada moneda en la propuesta 2.

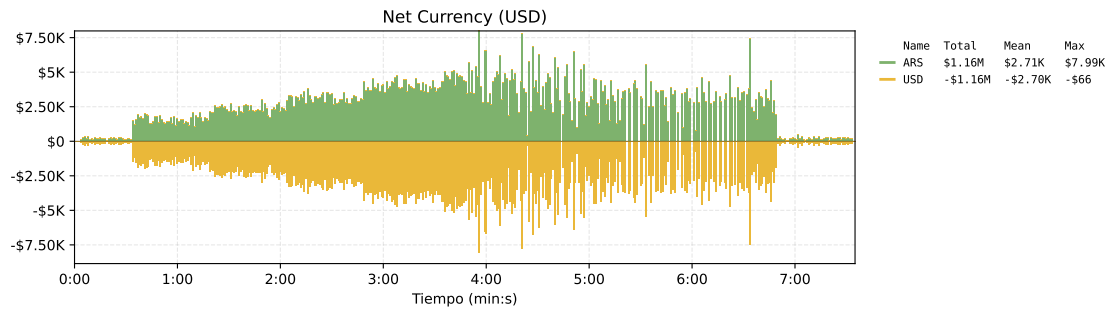


Figura 99. Neto operado en cada moneda en la propuesta 2.

A.10. Resultados completos del escenario de resistencia sobre la propuesta 2

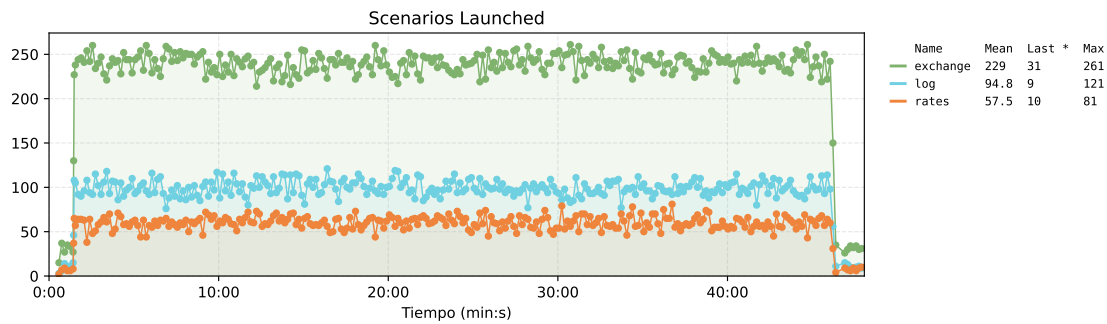


Figura 100. Escenarios de la propuesta 2.

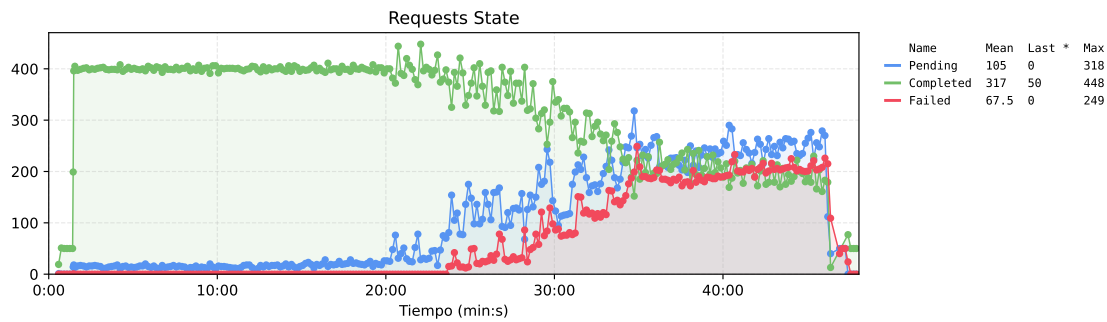


Figura 101. Estado de las requests de la propuesta 2.



Figura 102. Tiempo de respuesta de la propuesta 2.

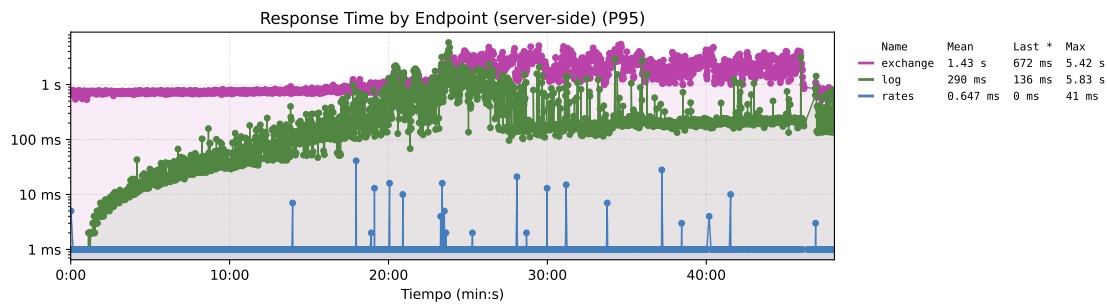


Figura 103. Tiempo de respuesta por endpoint de la propuesta 2. El eje tiene una escala logarítmica.

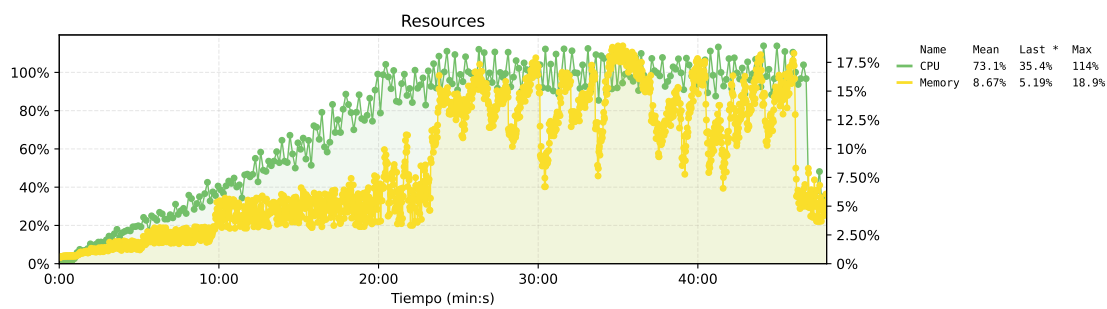


Figura 104. Estado de los recursos de la propuesta 2. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

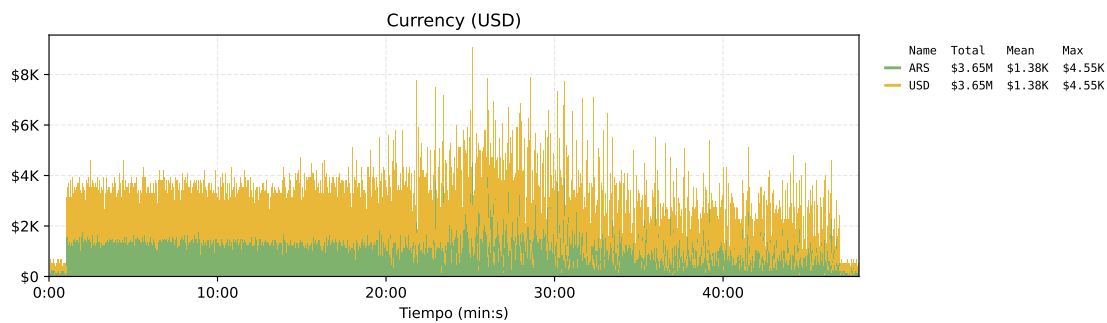


Figura 105. Volumen operado en cada moneda en la propuesta 2.

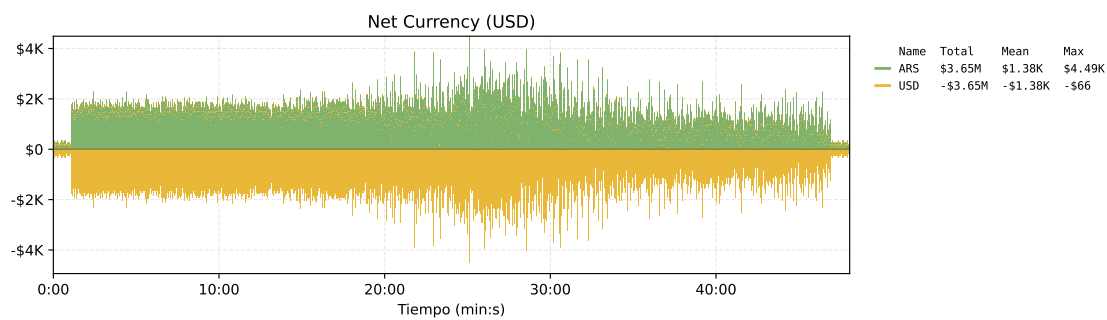


Figura 106. Neto operado en cada moneda en la propuesta 2.

A.11. Resultados completos del escenario de caos sobre la propuesta 2

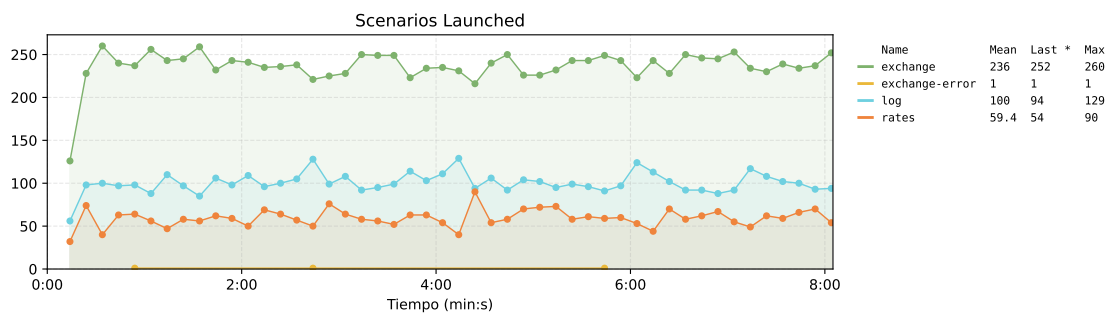


Figura 107. Escenarios de la propuesta 2.

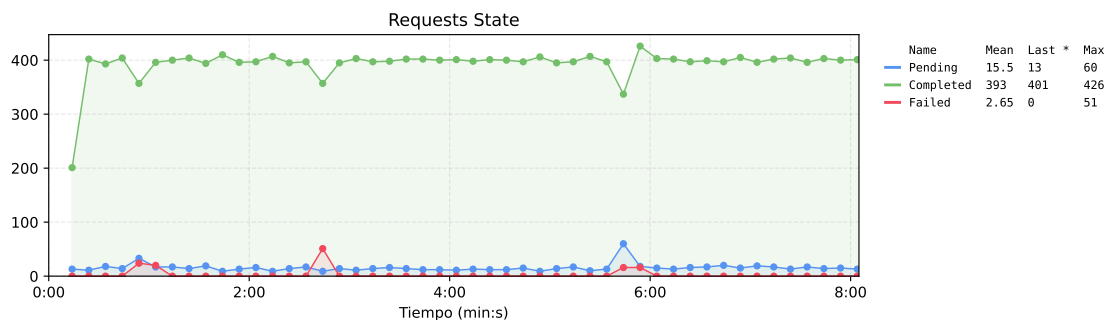


Figura 108. Estado de las requests de la propuesta 2.

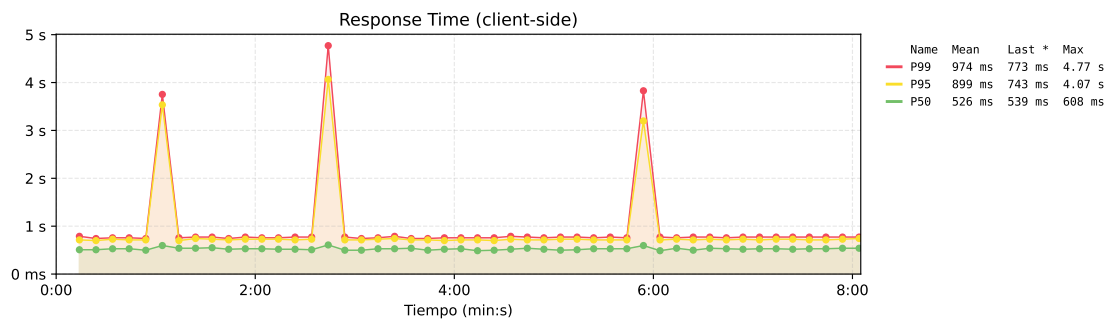


Figura 109. Tiempo de respuesta de la propuesta 2.

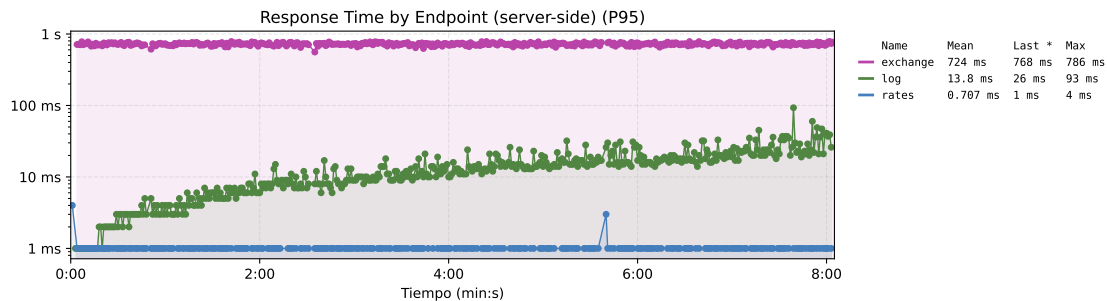


Figura 110. Tiempo de respuesta por endpoint de la propuesta 2. El eje tiene una escala logaritmica.

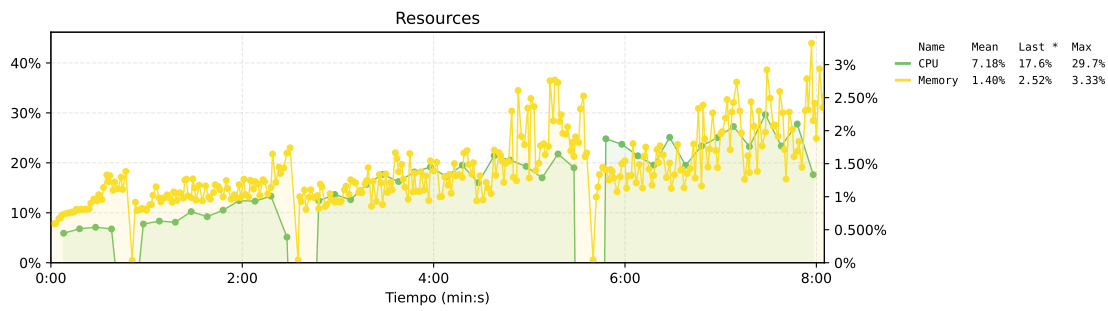


Figura 111. Estado de los recursos de la propuesta 2. El eje izquierdo muestra el uso de CPU, el eje derecho muestra el uso de memoria.

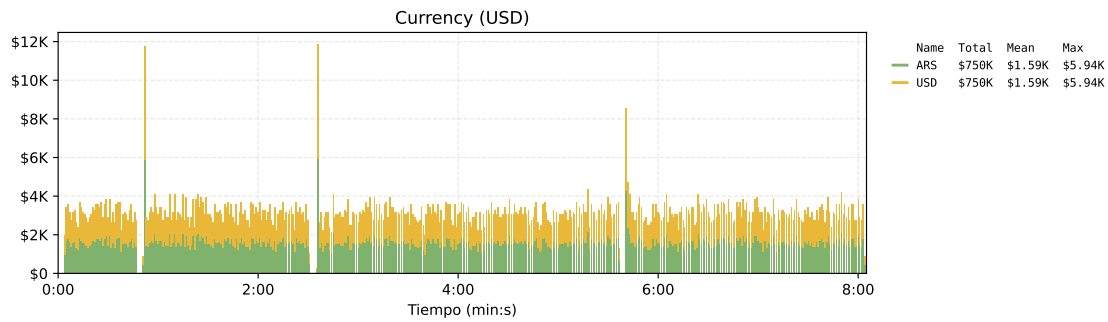


Figura 112. Volumen operado en cada moneda en la propuesta 2.

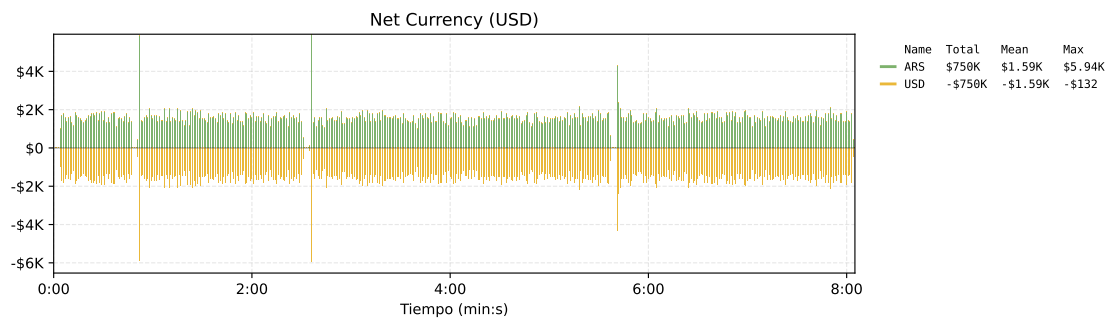


Figura 113. Neto operado en cada moneda en la propuesta 2.